

BÀI 1: GIỚI THIỆU

1. Mục tiêu

Thông qua bài này, sinh viên cần nắm rõ:

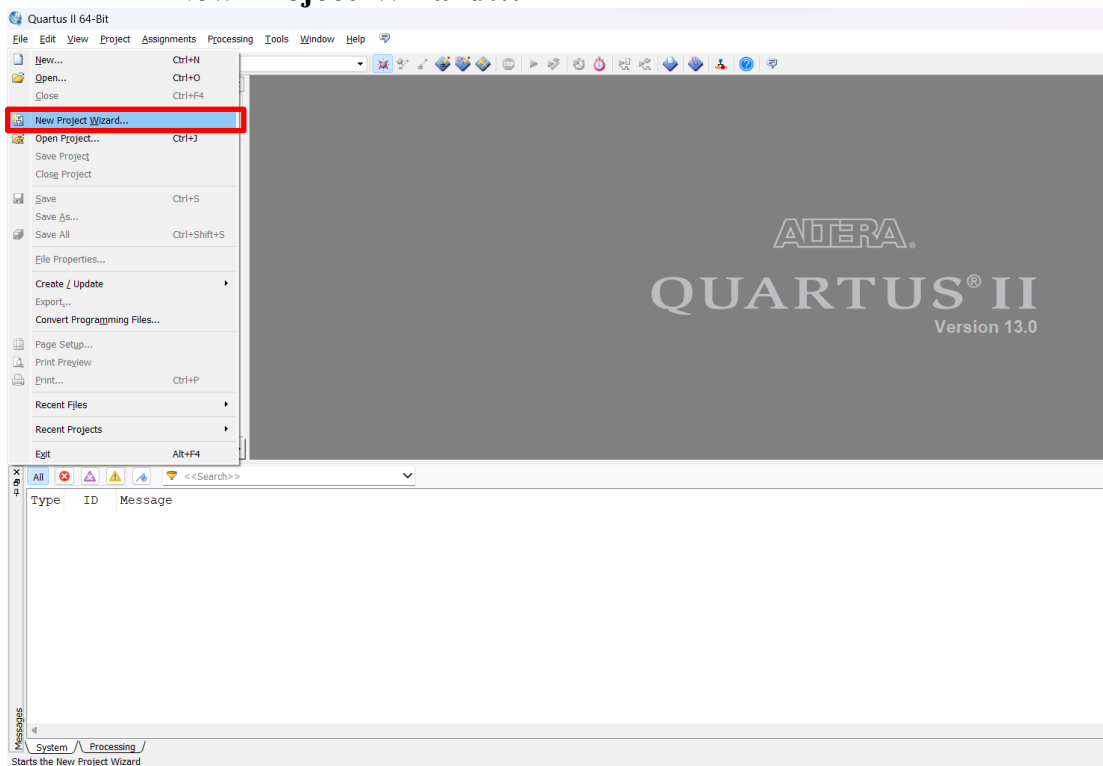
- Quy trình thiết kế hệ thống SoC gồm cả phần cứng và phần mềm.
- Cách sử dụng công cụ Quartus 13.0sp1 và Qsys để tạo phần cứng máy tính, tổng hợp và nạp xuống FPGA.
- Cách sử dụng công cụ NIOS II để tổng hợp và nạp phần mềm xuống máy tính.

2. Nội dung thực hành

2.1. Hệ thống phần cứng

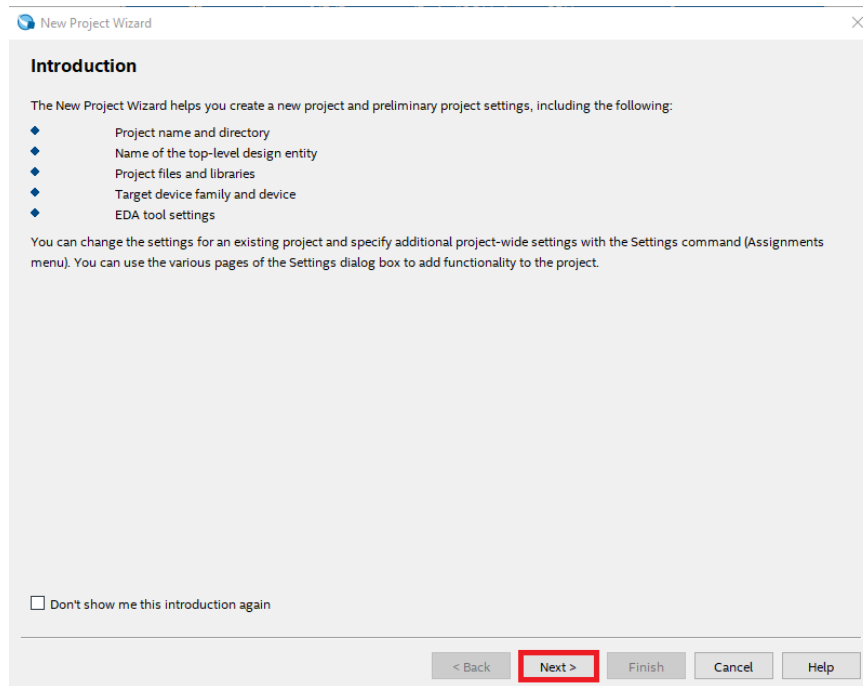
2.1.1. Tạo project Quartus

- Tạo thư mục ở ổ đĩa bất kì và đặt tên là **Bai1**.
- Tại cửa sổ giao diện của Quartus phiên bản 13.0, hình1, chọn **File** → **New Project Wizard...**



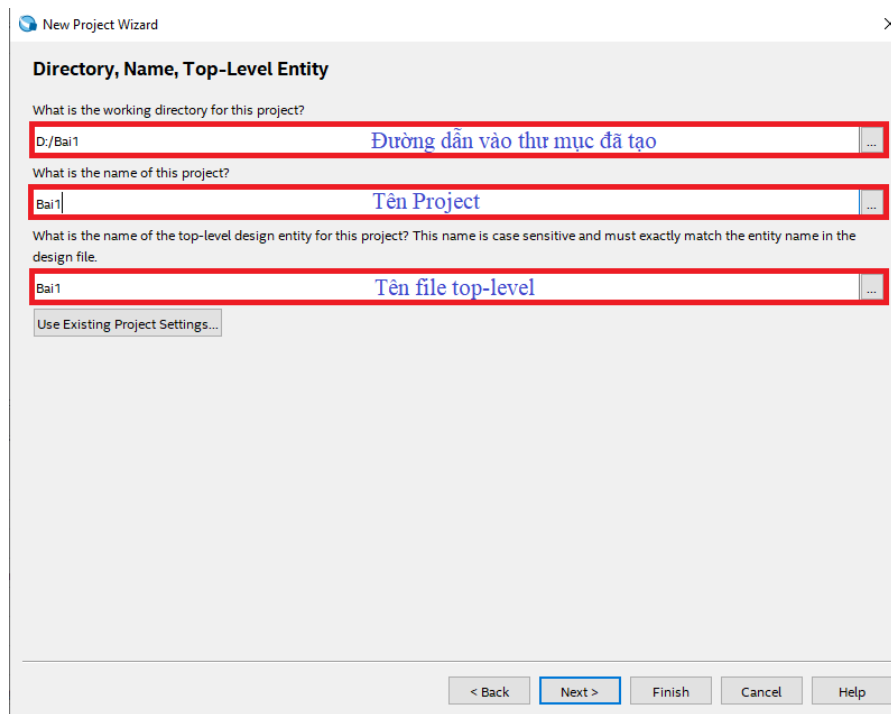
Hình 1. Giao diện phần mềm Quartus .

- Sau khi chọn như hình 1, một cửa sổ New Project Wizard hiện lên, tiếp tục chọn **Next** như hình 2.



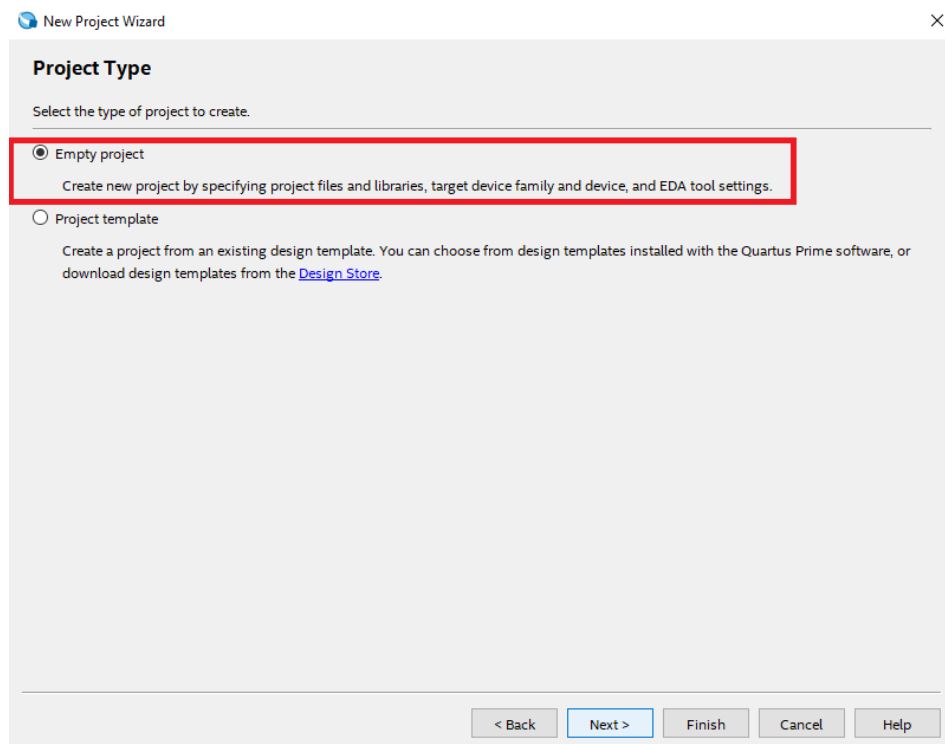
Hình 2. Tab Introduction.

- Ở tab **Directory, Name , Top-Level Entity**, chọn đường dẫn lưu project vào thư mục đã tạo trước đó và đặt tên như hình 3.



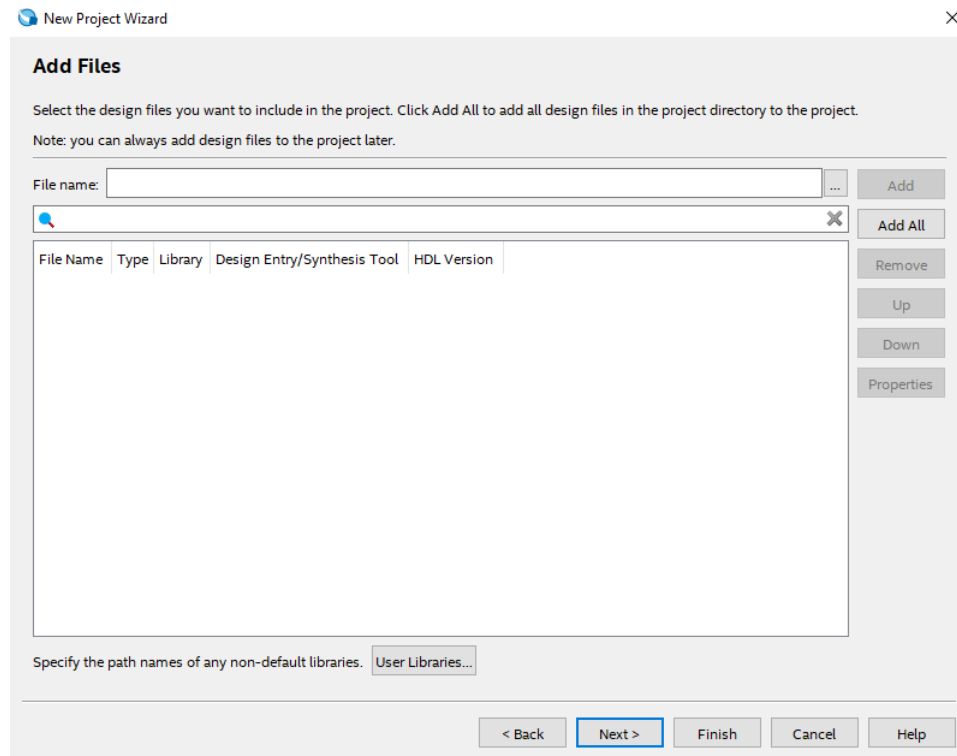
Hình 3. Tab Directory, Name, Top-Level Entity.

- Ở tab **Project Type**, chọn Empty project như hình 4 rồi nhấn **Next**.



Hình 4. Tab Project Type.

- Ở tab **Add Files**, chọn **Next** như hình 5.



Hình 5. Tab Add Files.

- Ở tab **Family, Device và Board Settings**,
 - Chọn Family là **Cyclone II** và device name là **EP2C35F672C6** hoặc **EP4E115F29C7** như hình 6 nếu dùng board **DE2**.
- Cuối cùng nhấn Finish.

New Project Wizard [page 3 of 5]

Select the family and device you want to target for compilation.
You can install additional device support with the Install Devices command on the Tools menu.

Device family

Family: Cyclone II
Devices: All

Target device

☐ Auto device selected by the Fitter
☒ Specific device selected in 'Available devices' list
☐ Other: n/a

Show in 'Available devices' list

Package: FBGA
Pin count: 672
Speed grade: Fastest
Name filter:
☒ Show advanced devices ☐ HardCopy compatible only

Available devices:

Name	Core Voltage	LEs	User I/Os	Memory Bits	Embedded multiplier 9-bit elements
EP2C35F672C6	1.2V	33216	475	483840	70
EP2C50F672C6	1.2V	50528	450	594432	172

Companion device

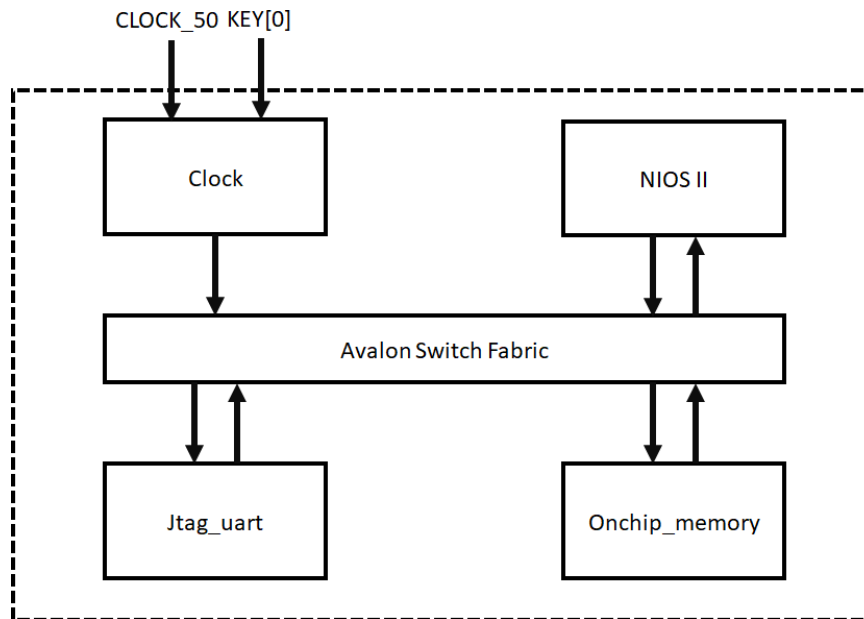
HardCopy:
☐ Limit DSP & RAM to HardCopy device resources

< Back Next > Finish Cancel Help

Hình 6. Tab Family, Device & Board Settings.

2.1.2. Xây dựng hệ thống phần cứng sử dụng Qsys.

a. Tổng quan hệ thống

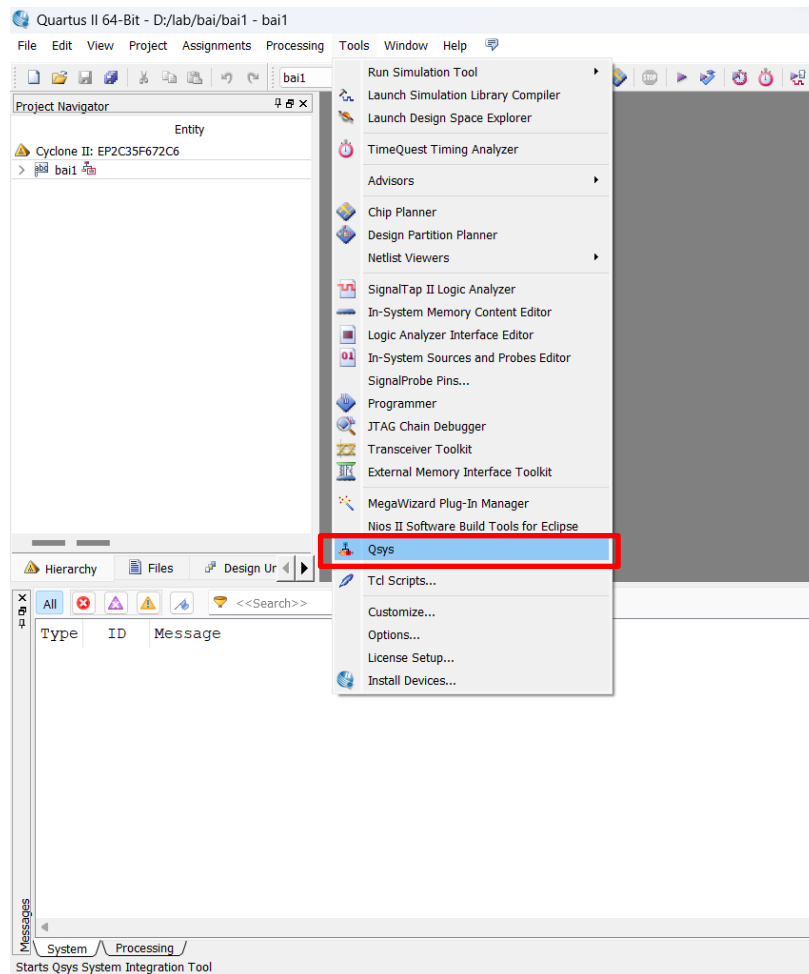


Hình 8. Tổng quan hệ thống.

- Hệ thống phần cứng xây dựng là một máy tính đơn giản, với các thành phần được thể hiện trong hình 7. Trong đó thành phần quan trọng nhất là NIOS II đóng vai trò là CPU và Onchip_memory là bộ nhớ để lưu trữ dữ liệu của hệ thống.

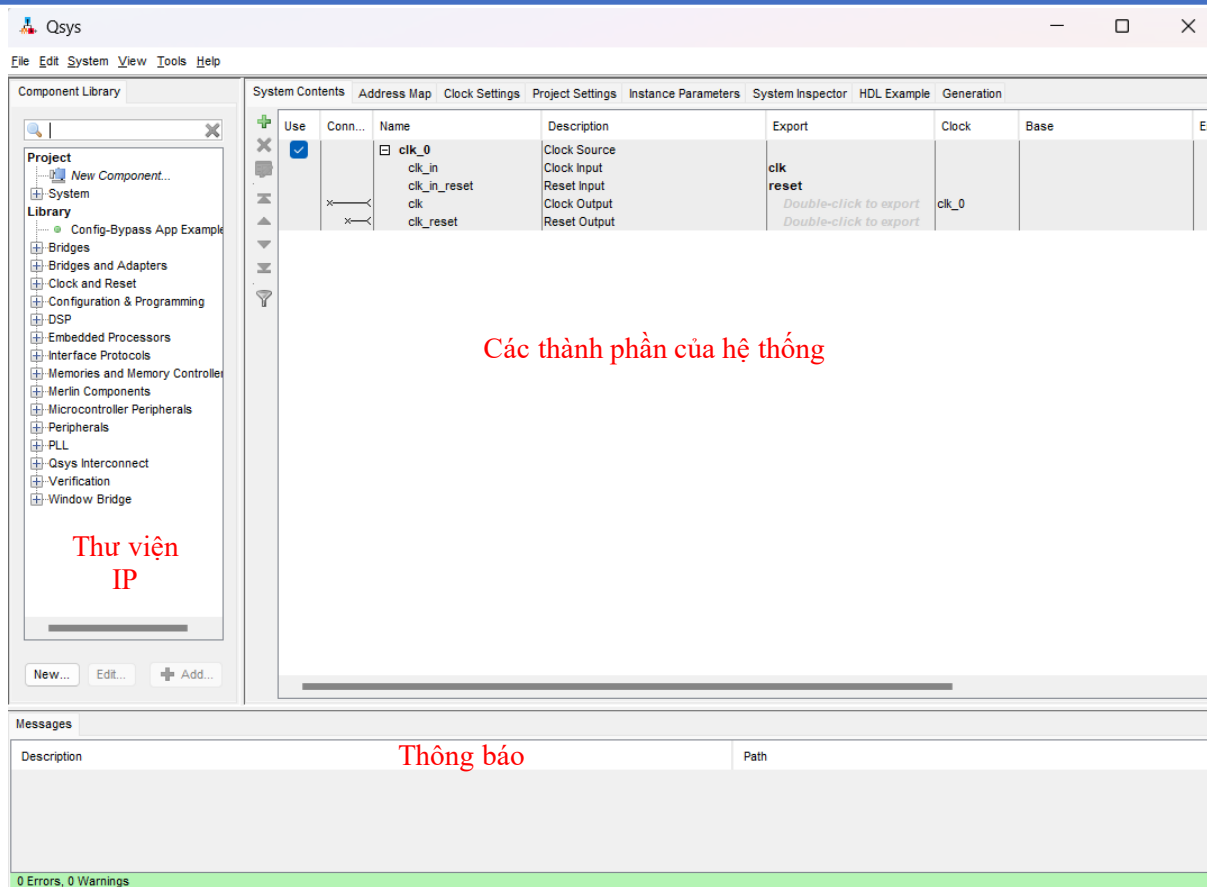
b. Tiến hành xây dựng hệ thống trên Qsys

- Ở cửa số 3 của phần mềm Quartus Prime, chọn **Tools** → **Qsys** như hình 9.



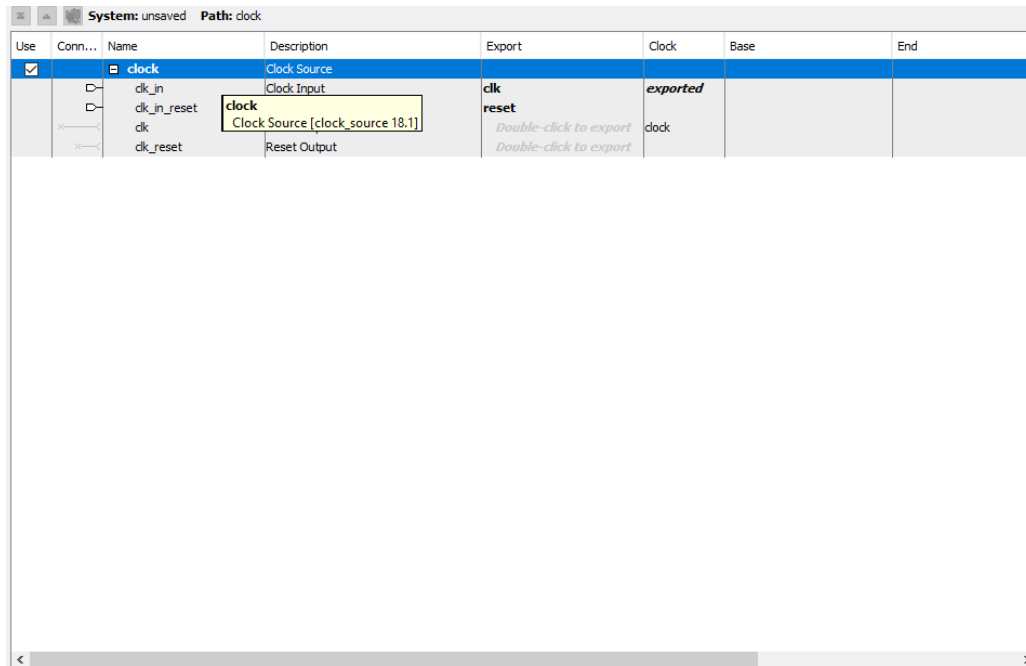
Hình 9. Mở Qsys.

- Sau khi mở phần mềm Qsys, sẽ có được cửa sổ như hình 10.



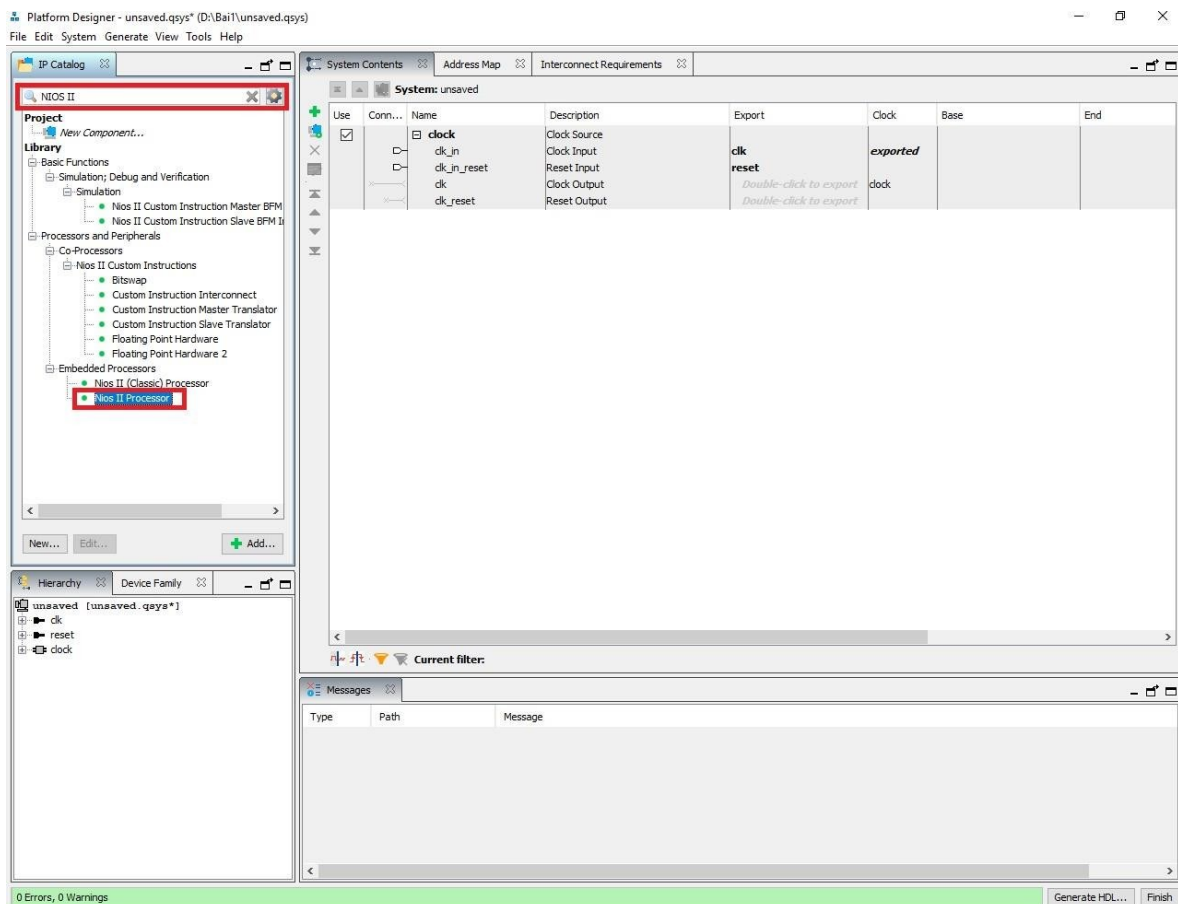
Hình 10. Giao diện phần mềm Platform Designer.

- Trong đó, ở tab System contents đã có thành phần Clock trong hệ thống, tiến hành đổi tên clk_0 thành Clock. (Bấm chuột phải vào IP clk_0 và chọn Rename). Sau khi đổi tên thành công, sẽ được như hình 11.



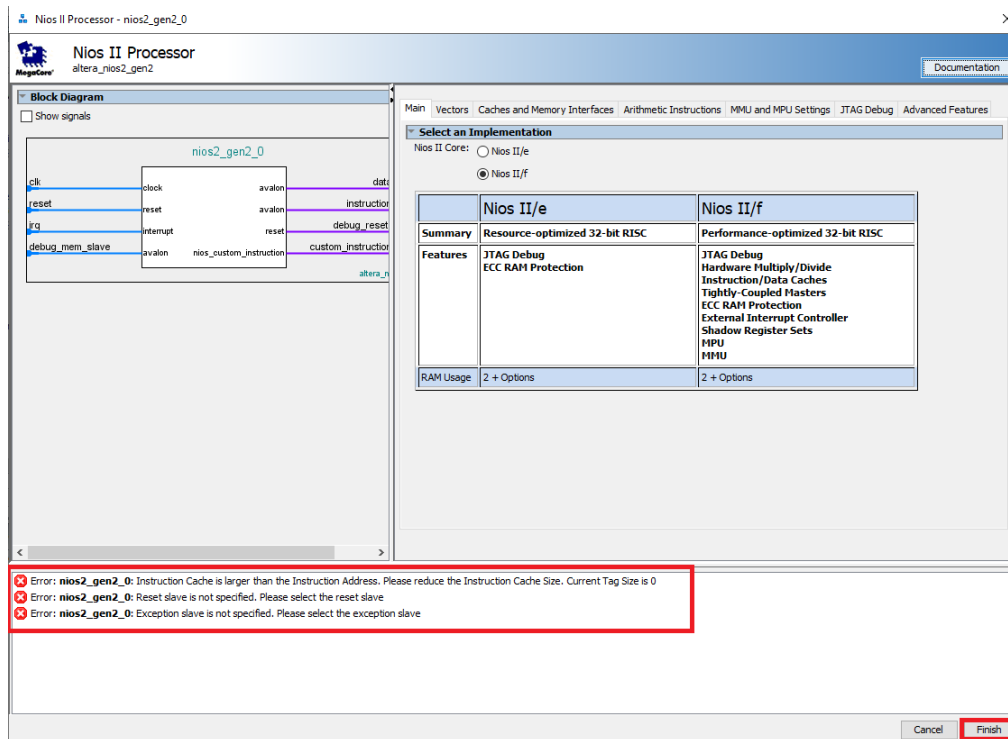
Hình 10. Tab System Contents.

- Tiếp theo, thêm CPU NIOS II bằng cách gõ NIOS II vào khung tìm kiếm và chọn như hình 12. Thêm vào hệ thống bằng cách bấm đúp chuột trái.



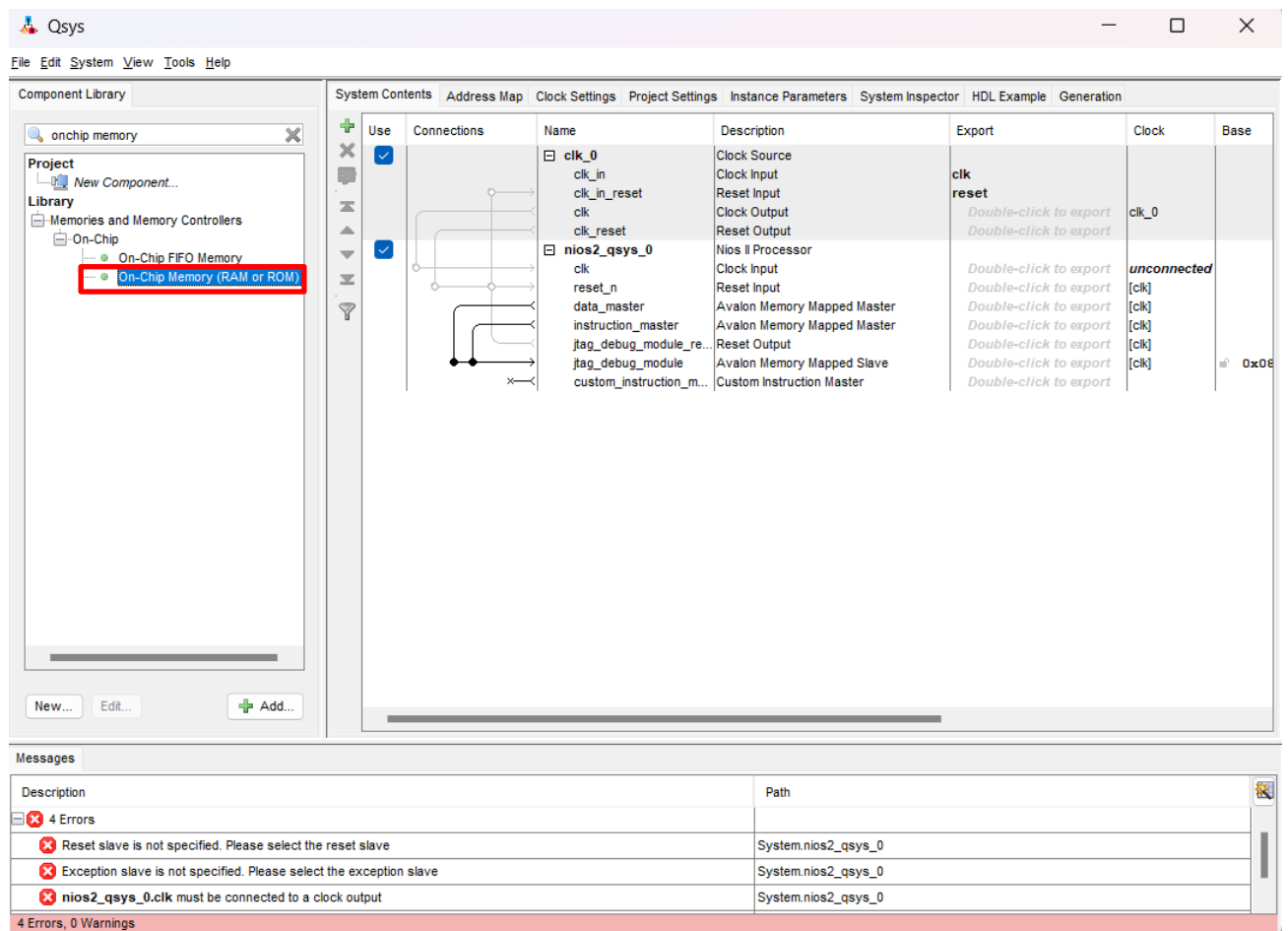
Hình 12. Thêm Nios II.

- Sau khi thêm vào hệ thống, sẽ có một hộp thoại mở ra như hình 13. Trong hình 13 sẽ hiện lỗi, nhưng sinh viên không quan tâm và bấm Finish, việc sửa lỗi sẽ thực hiện sau.



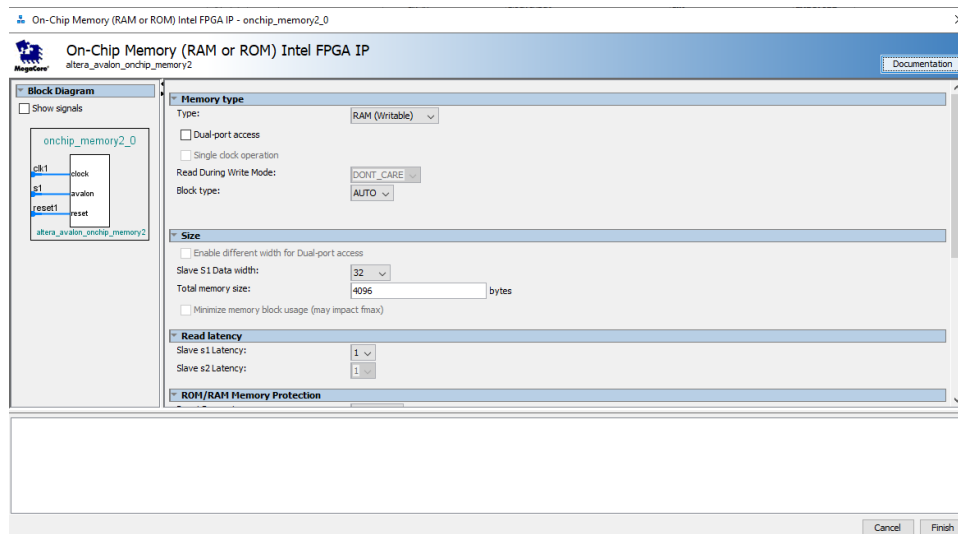
Hình 13. Cửa sổ cấu hình Nios II.

- Tiếp theo, thêm Onchip_memory bằng cách gõ onchip memory vào ô tìm kiếm và chọn như hình 14. Ngoài ra, ở hình 14, bên tab **System Contents** sẽ thấy sự xuất hiện của NIOS II đã thêm trước đó.



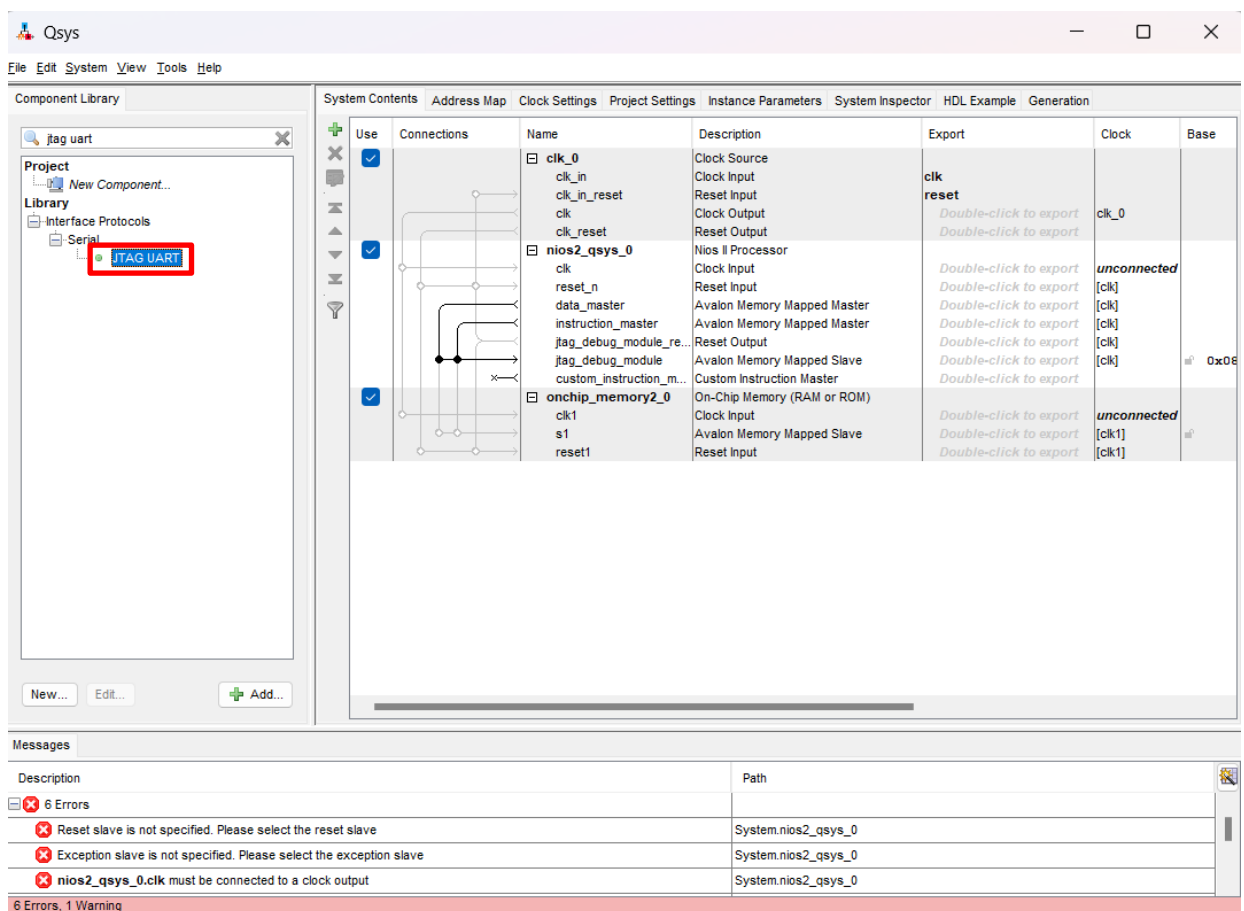
Hình 14. Thêm Onchip-Memory.

- Sau khi thêm onchip memory, cửa sổ hiện ra như hình 15.

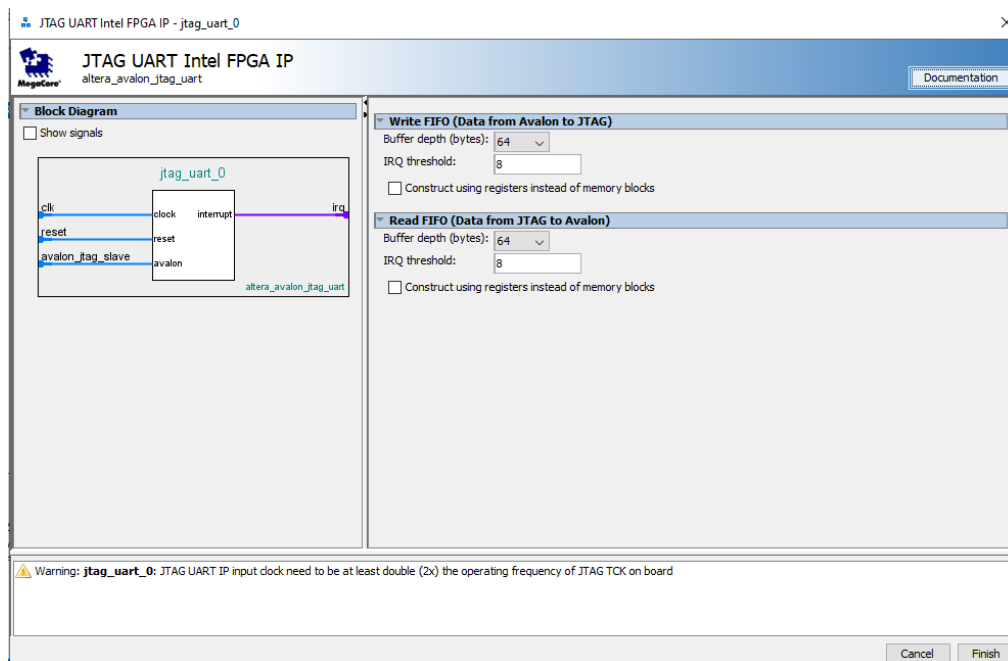


Hình 15. Cửa sổ cấu hình On-Chip Memory.

- Tương tự, thêm thành phần jtag-uart như hình 16 và cửa sổ của mô đun jtag-uart như hình 16.

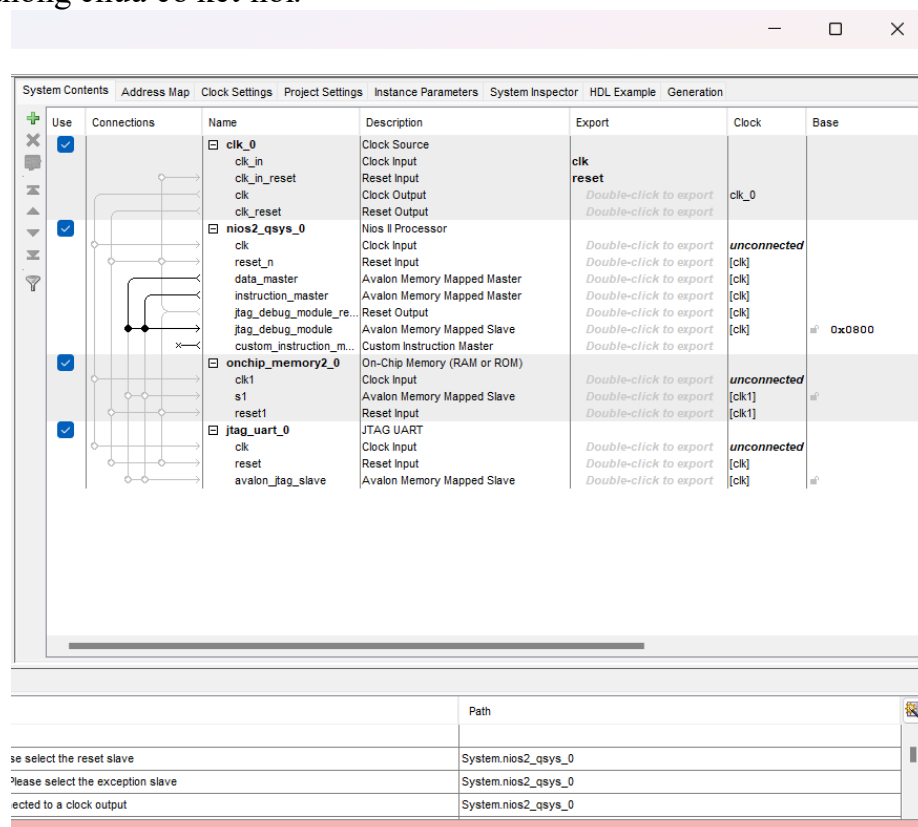


Hình 16. Thêm Jtag-uart.



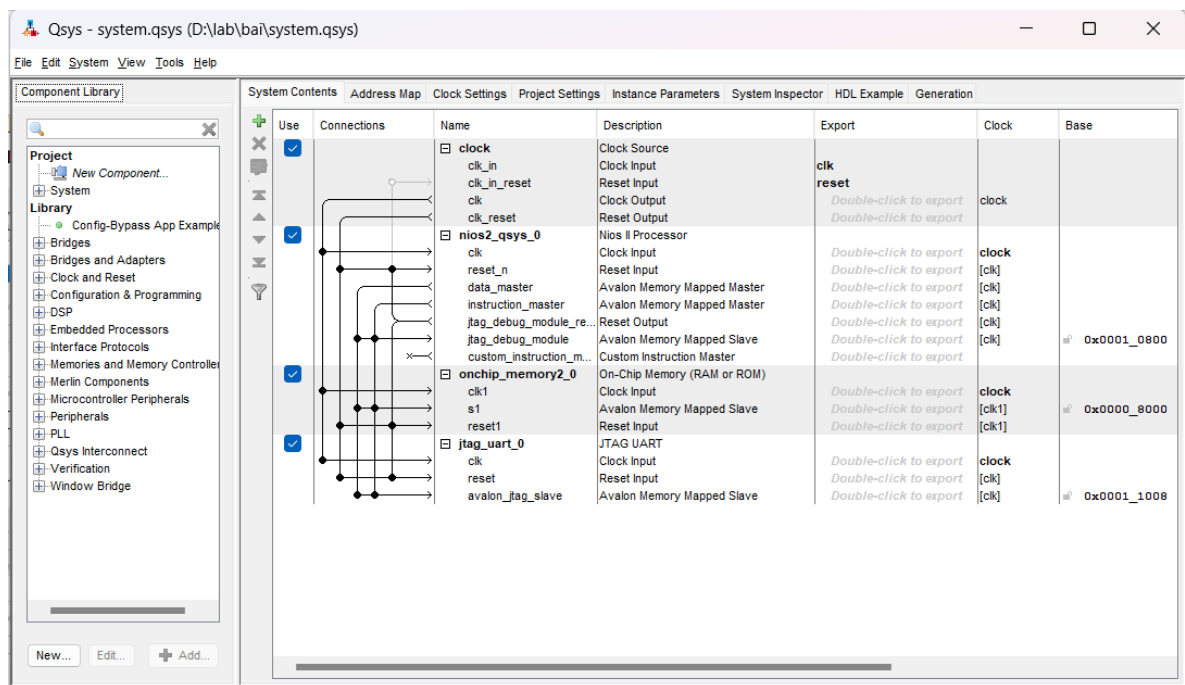
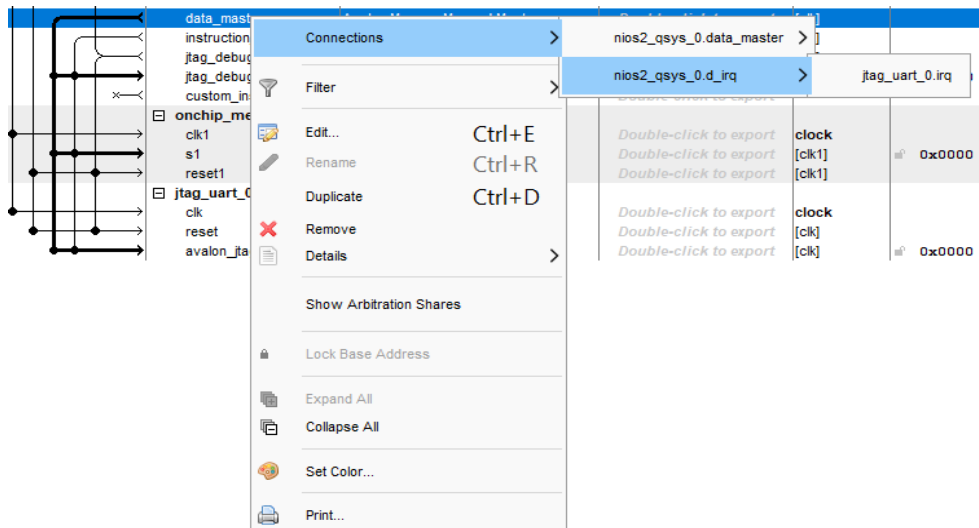
Hình 17. Cửa sổ cấu hình Jtag-uart.

- Sau khi thêm tất cả các mô đun theo yêu cầu, sẽ được hệ thống như hình 18, đây là hệ thống chưa có kết nối.



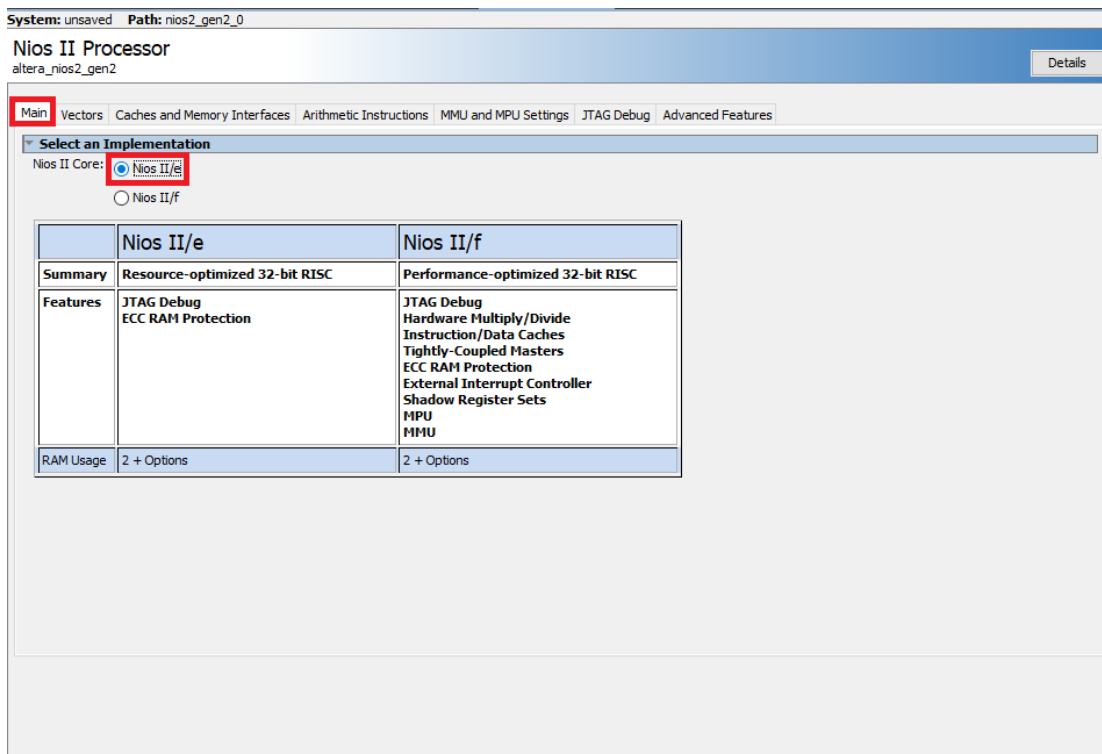
Hình 18. Hệ thống chưa kết nối các tín hiệu.

- Sau đó, tiến hành kết nối cho các thành phần như hình 19.
- Kết nối bằng cách chuột phải vào từng thành phần để nối



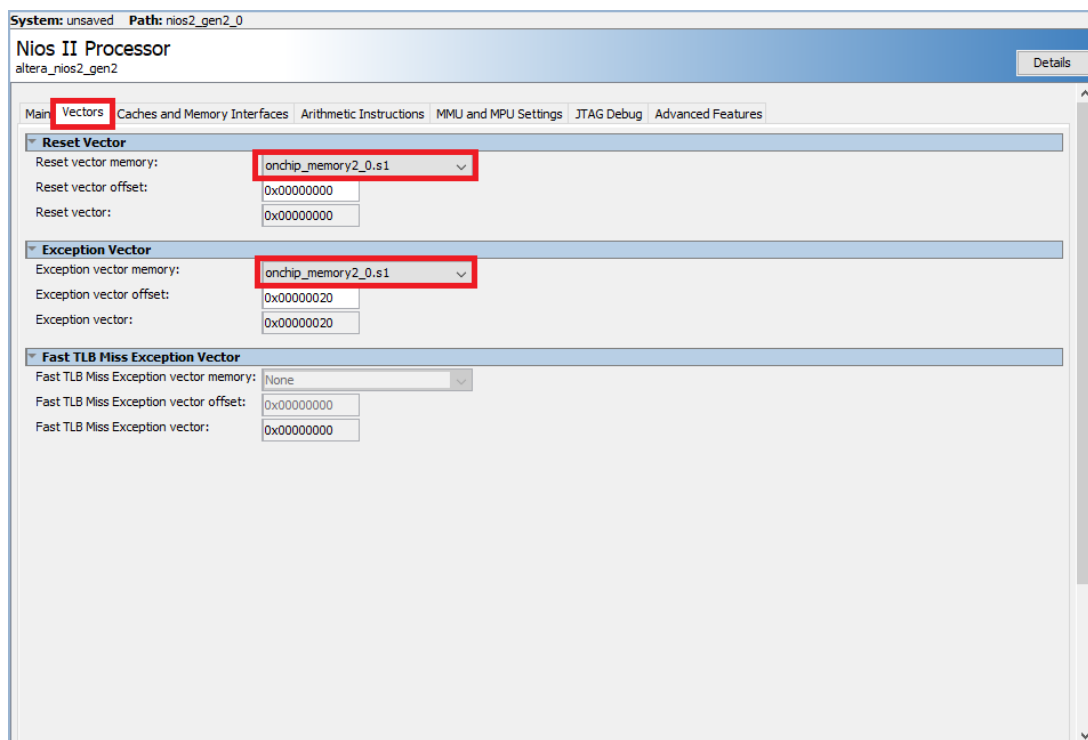
Hình 19. Hệ thống đã kết nối các tín hiệu.

- Tiếp theo tiến hành cấu hình cho từng thành phần. Đầu tiên, cấu hình cho NIOS II bằng cách bấm đúp chuột trái vào mô đun nios_gen2_0 và cấu hình ở cửa sổ Parameters. Ở tab Main, chọn Nios II e như hình 20.



Hình 20. Cửa sổ cấu hình Nios II, tab Main.

- Tiếp đến, chuyển sang tab Vectors và cấu hình như hình 21.



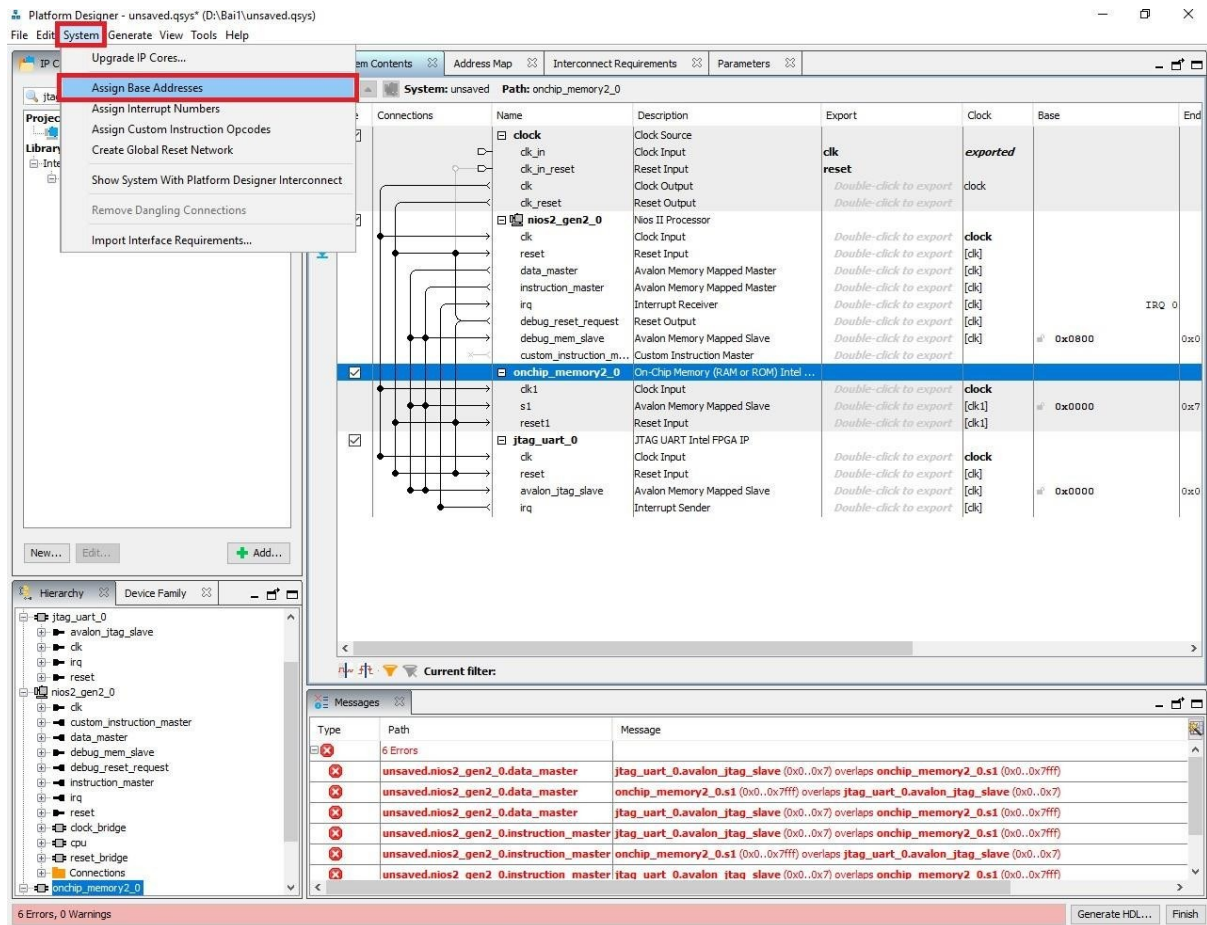
Hình 21. Cửa sổ cấu hình Nios II, tab Vectors.

- Tiếp tục là cấu hình Onchip_memory2_0. Tương tự, bấm đúp chuột trái vào onchip_memory2_0 và cấu hình kích thước bộ nhớ là **32768 bytes** như hình 22.

The screenshot shows the configuration window for the On-Chip Memory (RAM or ROM) Intel FPGA IP. The window is titled "On-Chip Memory (RAM or ROM) Intel FPGA IP" and "altera_avalon_onchip_memory2". The "Memory type" section shows "Type" set to "RAM (Writable)". The "Size" section is highlighted with a red box, showing "Slave S1 Data width" set to "32" and "Total memory size" set to "32768 bytes". The "Read latency" section shows "Slave s1 Latency" and "Slave s2 Latency" both set to "1". The "ROM/RAM Memory Protection" section shows "Reset Request" set to "Enabled". The "ECC Parameter" section shows "Extend the data width to support ECC bits" set to "Disabled". The "Memory initialization" section shows "Initialize memory content" checked and "Enable non-default initialization file" unchecked.

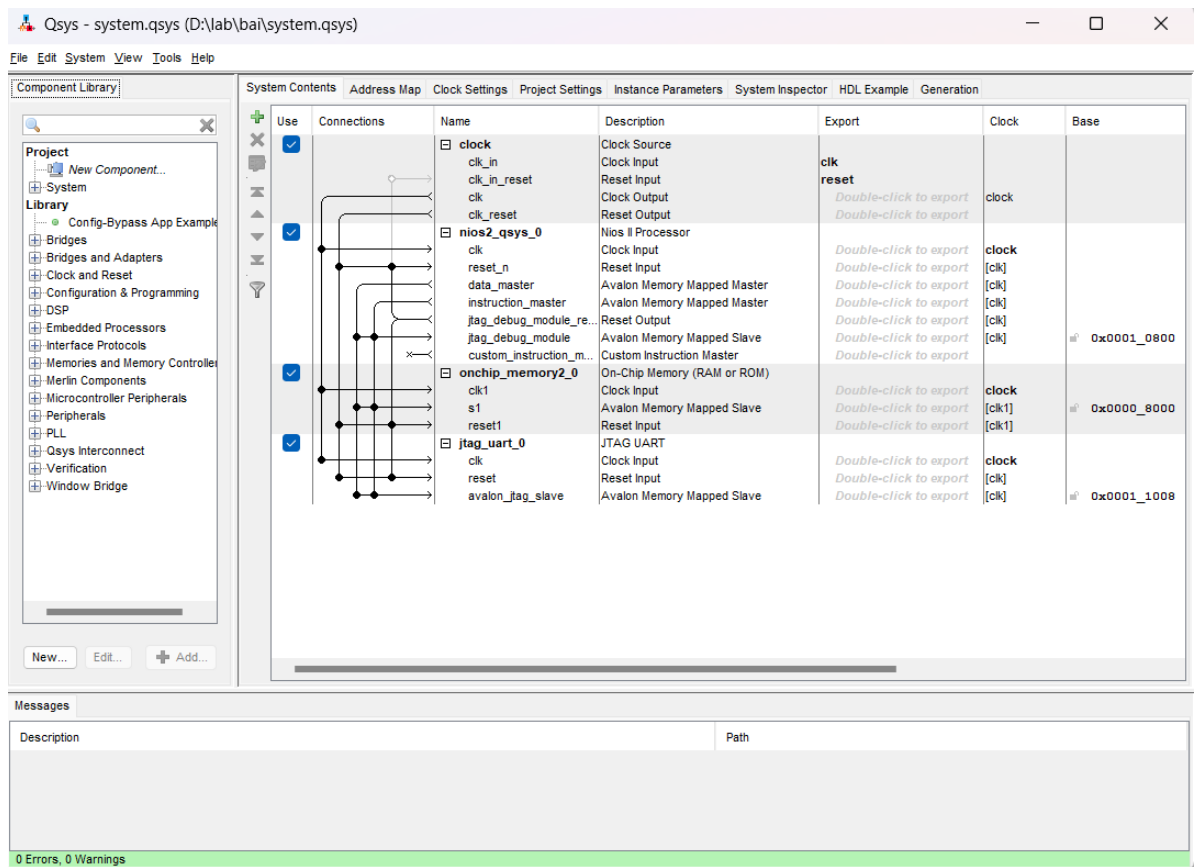
Hình 22. Cửa sổ cấu hình On-Chip Memory.

Cuối cùng, gán địa chỉ lại cho các thành phần trong hệ thống. Chọn **System** → **Assign Based Addresses**.



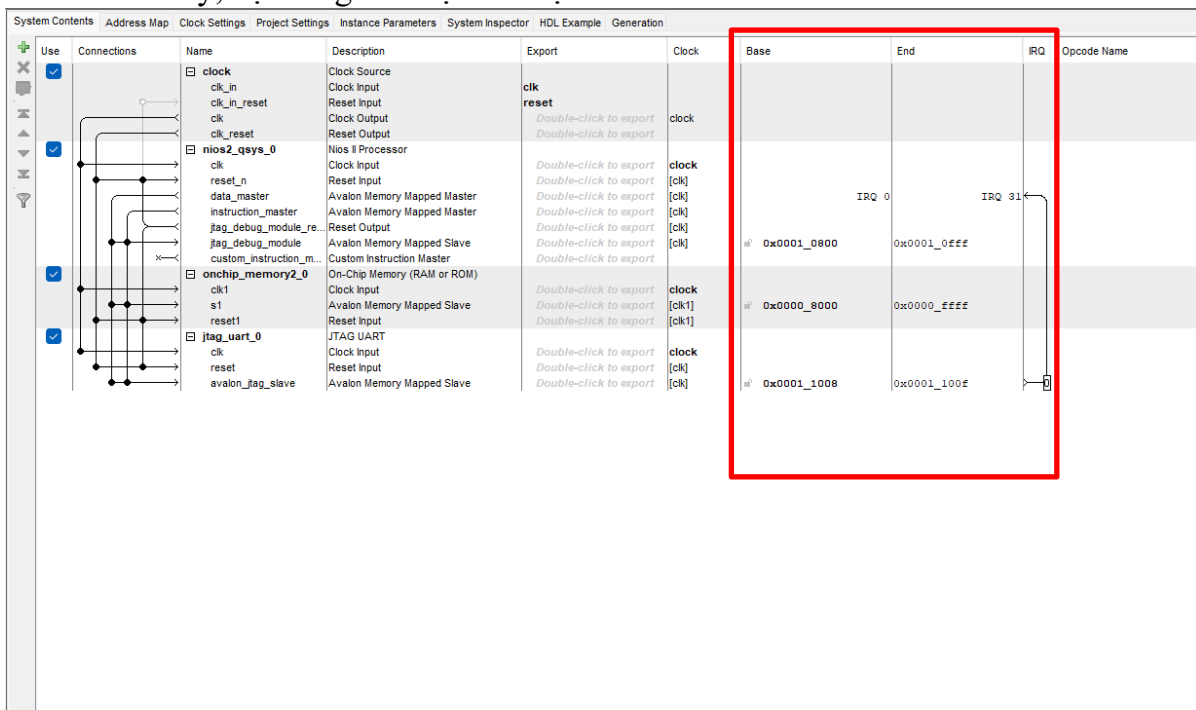
Hình 23. Hệ thống chưa gán địa chỉ.

- Sau khi gán địa chỉ tự động cho hệ thống, sẽ được như hình 24 với không còn lỗi.



Hình 24. Hệ thống sau khi gán địa chỉ.

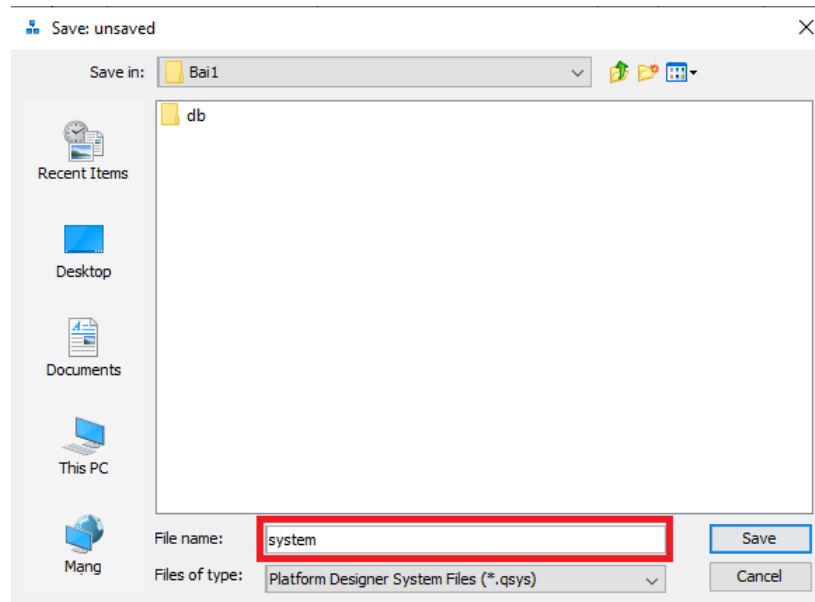
- Chú ý, hệ thống có được các địa chỉ như hình bên 25.



Hình 25. Địa chỉ của các thành phần.

**** Lưu ý:** Các địa chỉ này sẽ không cố định và có thể bài làm của sinh viên sẽ không giống với bài hướng dẫn.

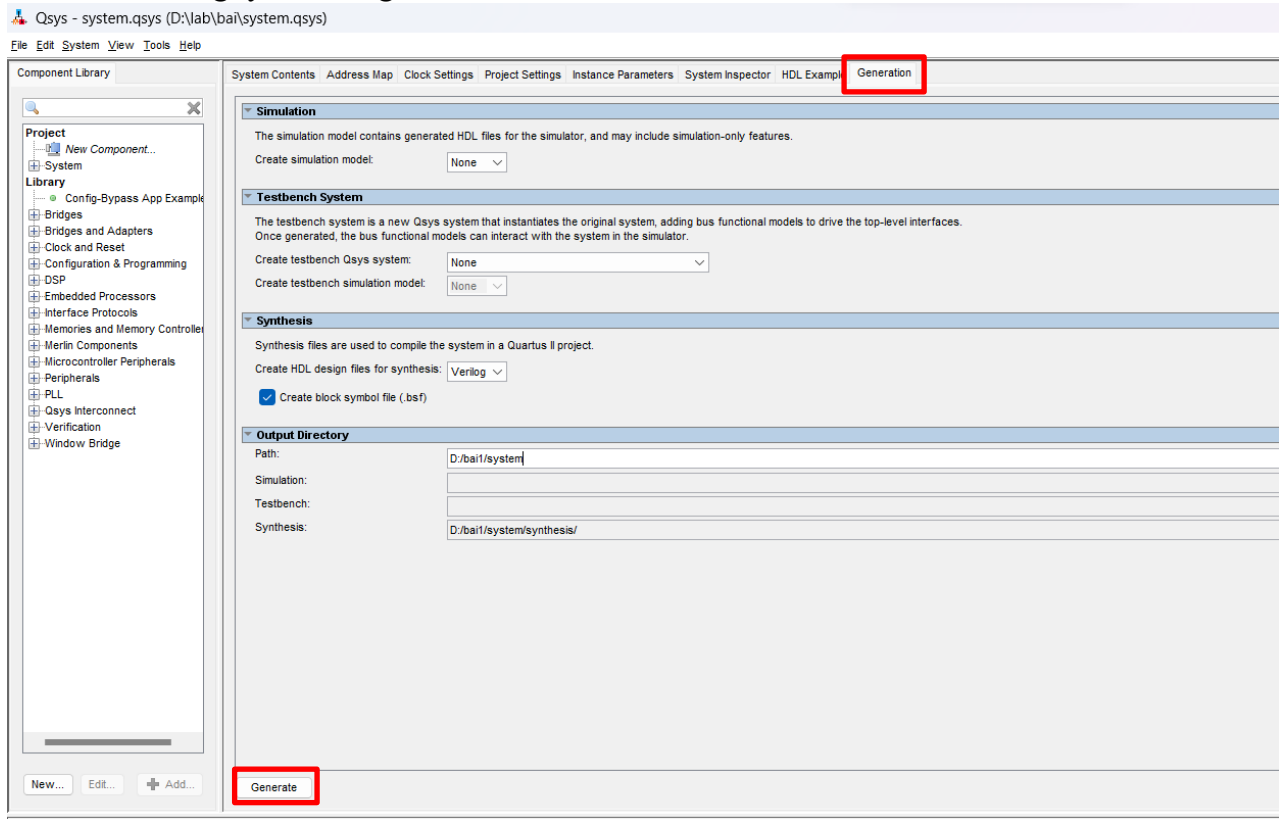
- Cuối cùng, lưu hệ thống với tên là system. Chọn **File** → **Save (hoặc Ctrl + S)** và chọn như hình 26.



Hình 26. Lưu hệ thống.

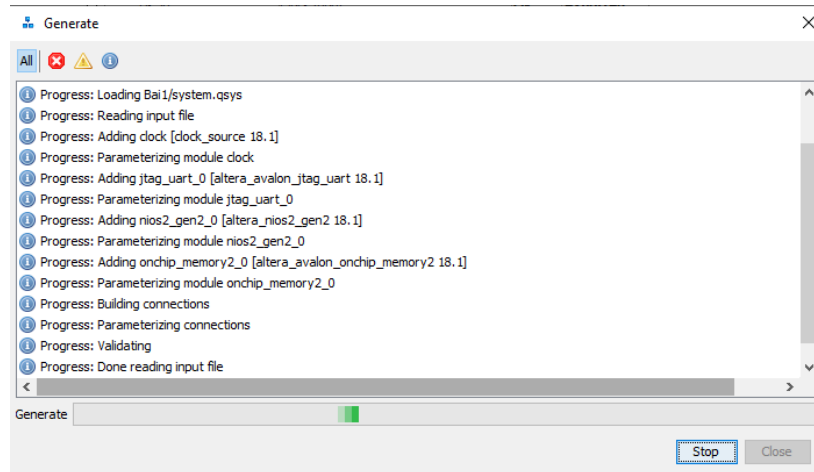
- Sau khi lưu hoàn tất, tạo ra hệ thống phần cứng bằng cách chọn **Generate HDL** như hình 28.

Giữ nguyên đường dẫn như hình 28 và chọn **Generate**.



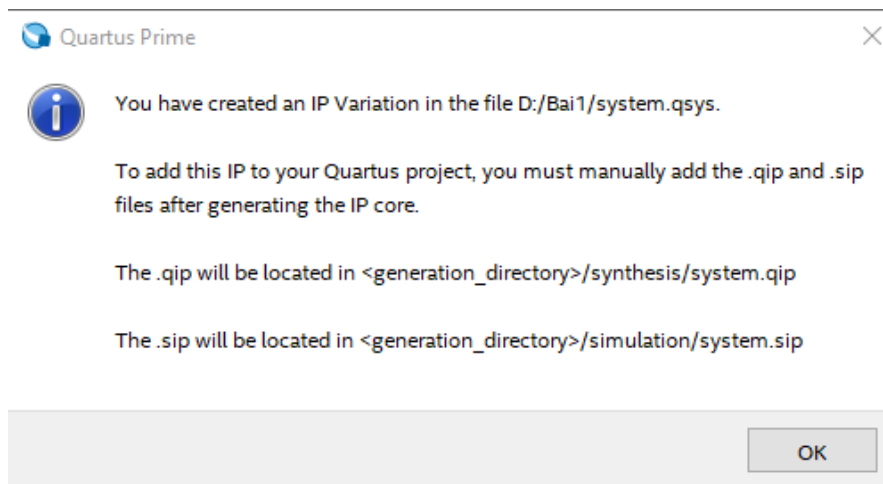
Hình 28. Generate hệ thống.

- Khi chọn **Generate**, sẽ có hộp thoại hiện ra thông báo quá trình generate, hình 29.



Hình 29. Cửa sổ thông báo quá trình generate.

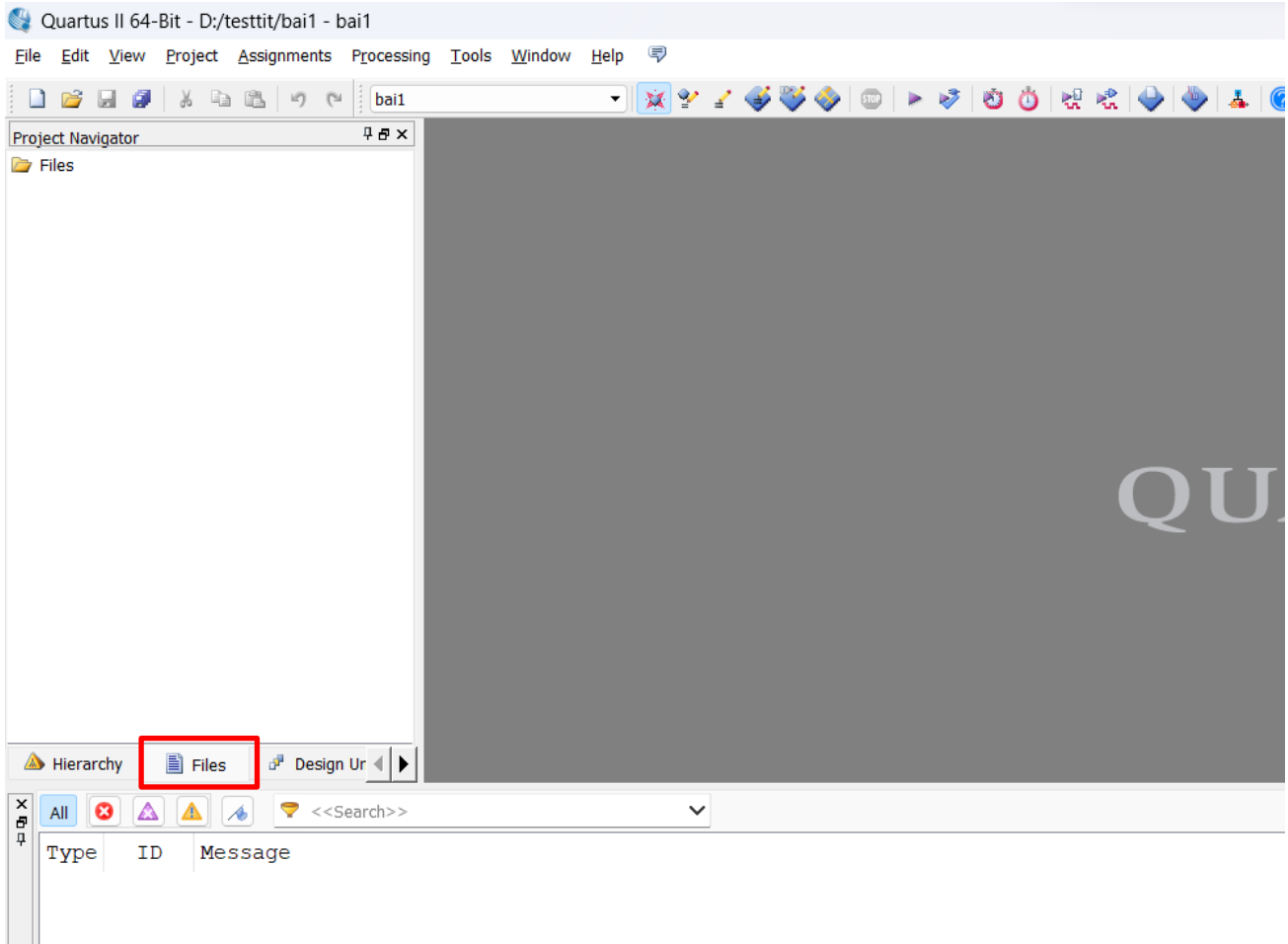
- Sau khi generate hoàn tất, chọn Finish. Sau khi chọn **Finish**, 1 hộp thoại sẽ hiện ra và thông báo đường dẫn và cách thêm hệ thống vừa tạo vào Quartus , hình 30.



Hình 30. Thông báo đường dẫn và cách thêm mô đun.

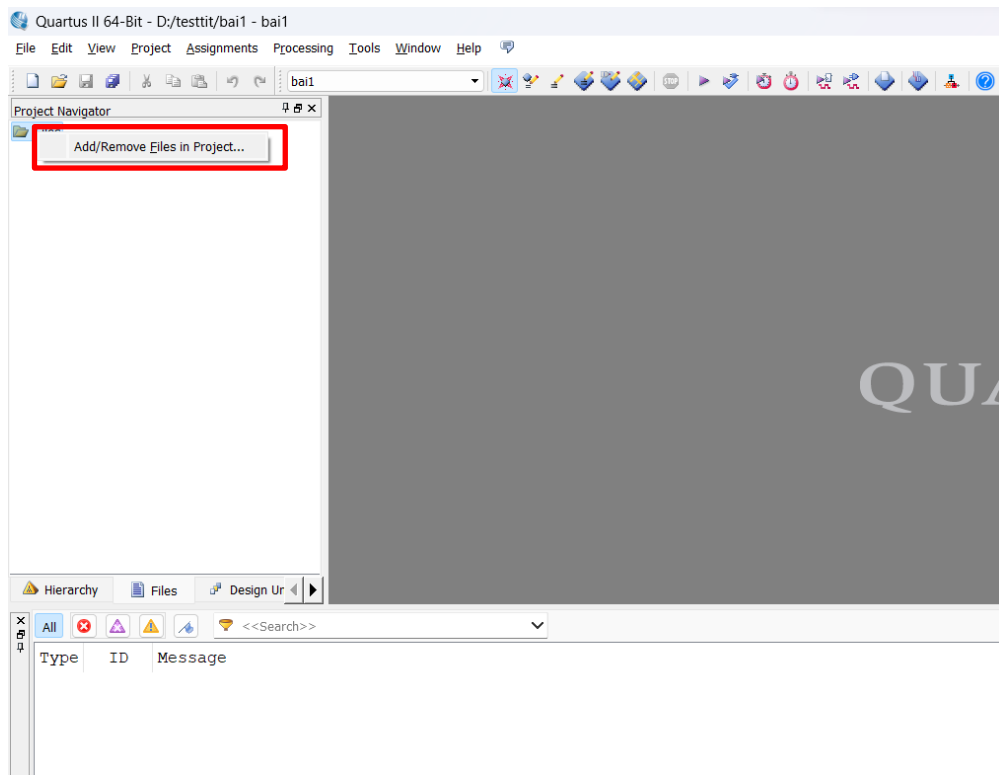
2.1.3. Tích hợp hệ thống vào Quartus Prime

- Tại cửa sổ Quartus, chọn như hình 31 để chuyển từ tab **Hierarchy** sang tab **Files**.



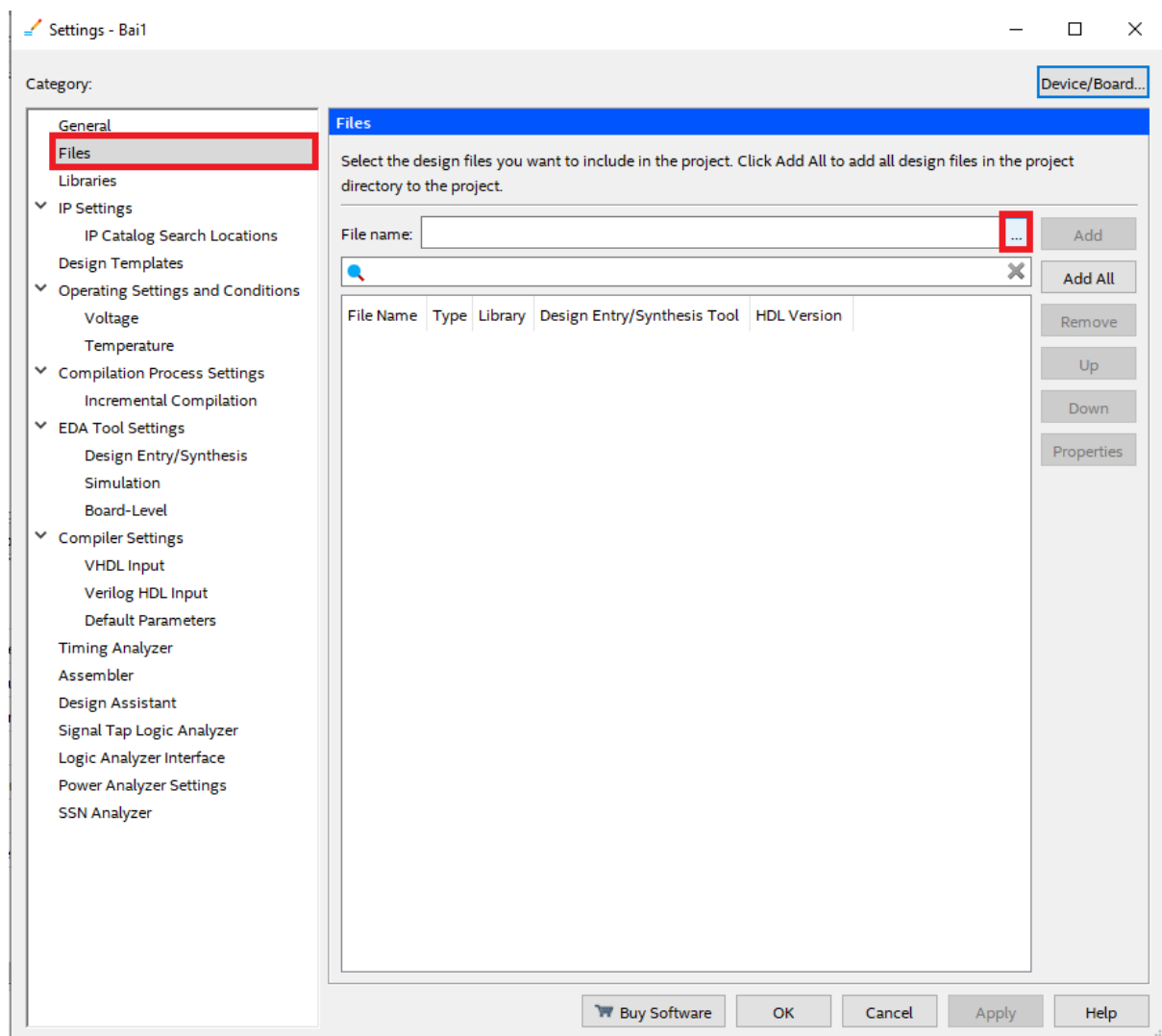
Hình 31. Chuyển sang tab Files ở giao diện Quartus .

- Ở tab Files, chọn chuột phải vào Files và chọn **Add/Remove Files in Project** như hình 32.



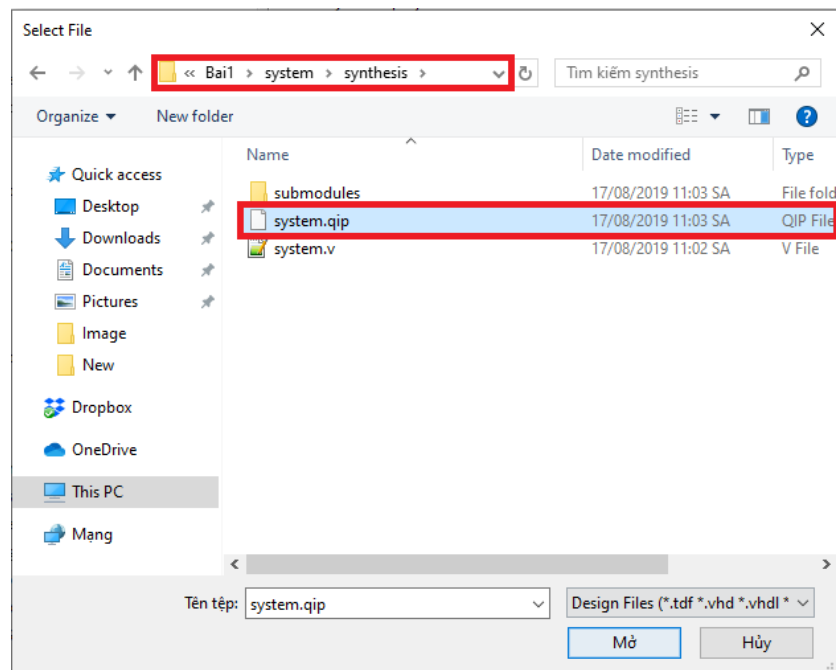
Hình 32. Thêm *system.qip* vào *project*.

- Ở cửa sổ Setting, chọn nút ... như hình 33 để dẫn đường dẫn file **system.qip** vào project.



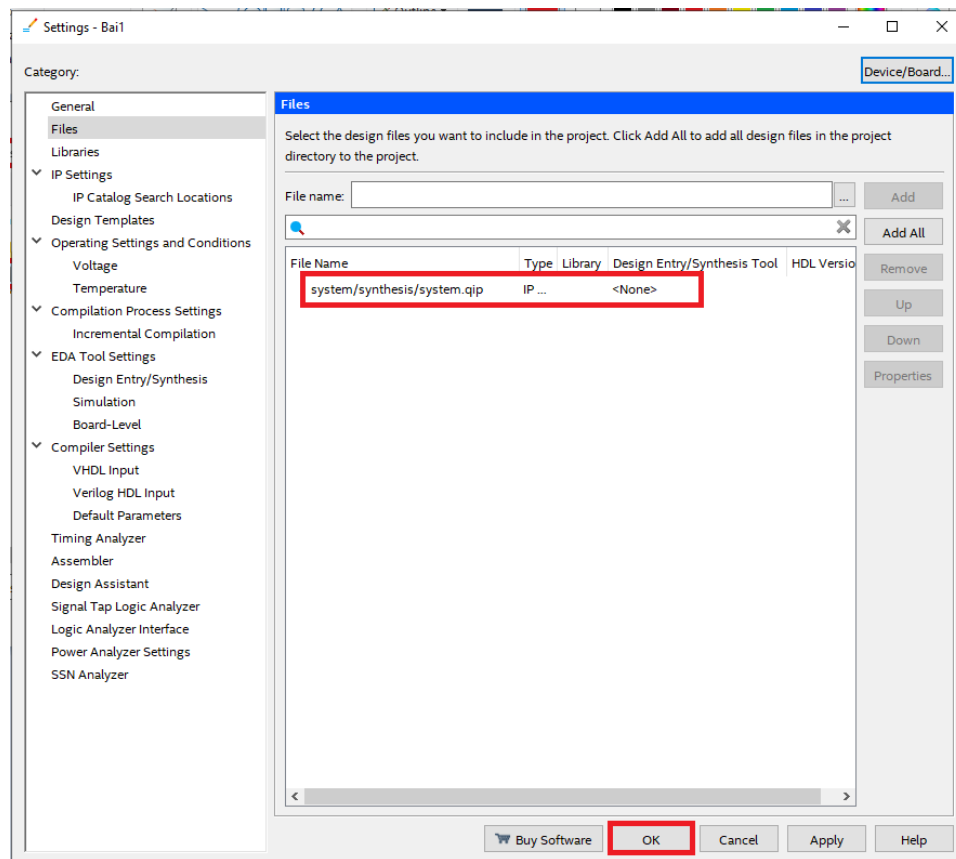
Hình 33. Cửa sổ Setting.

- File system.qip được tạo ra ở thư mục có đường dẫn: */system/synthesis*. Tiến hành thêm file **system.qip** như hình 34.



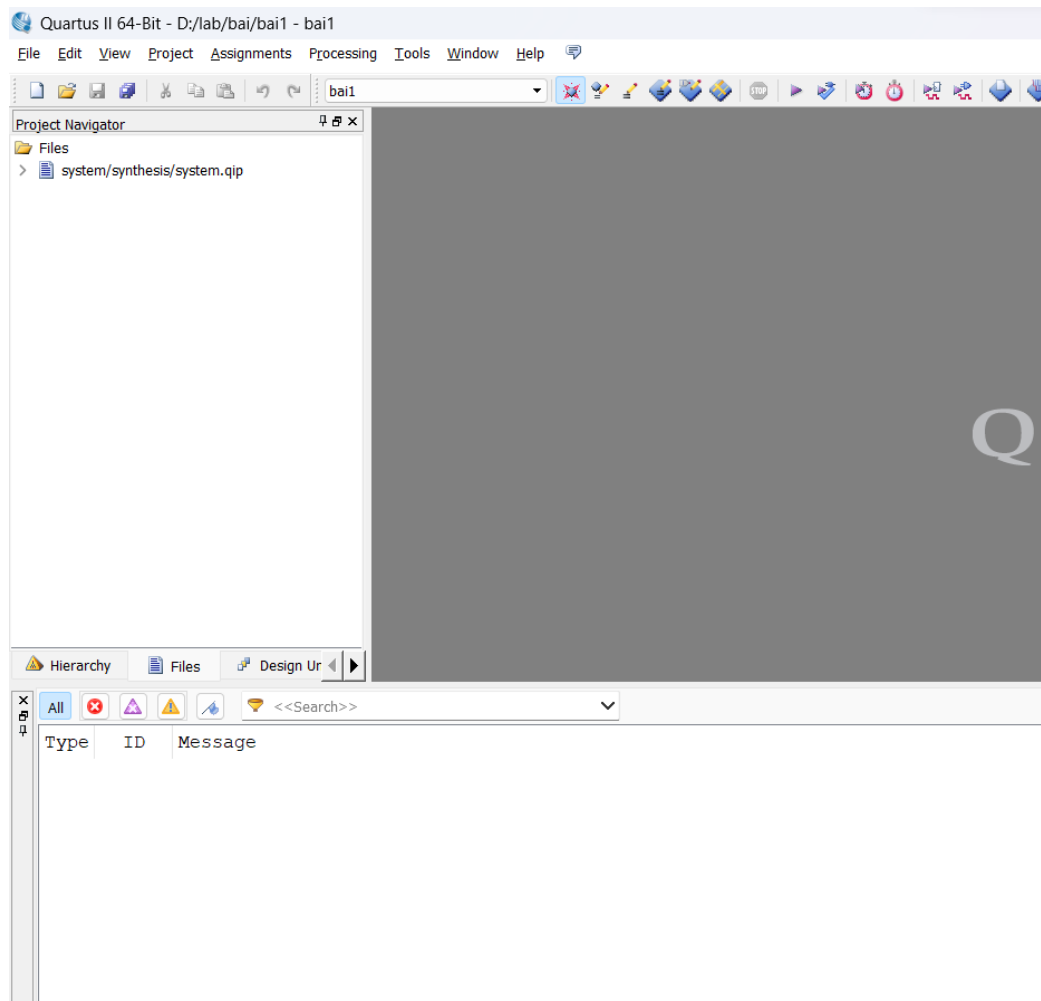
Hình 34. Cửa sổ Select File.

- Khi thêm ở giống như hình 35, sẽ trở về cửa sổ Setting và sẽ xuất hiện file system.qip như hình 35. Sau đó, chọn OK để hoàn tất.



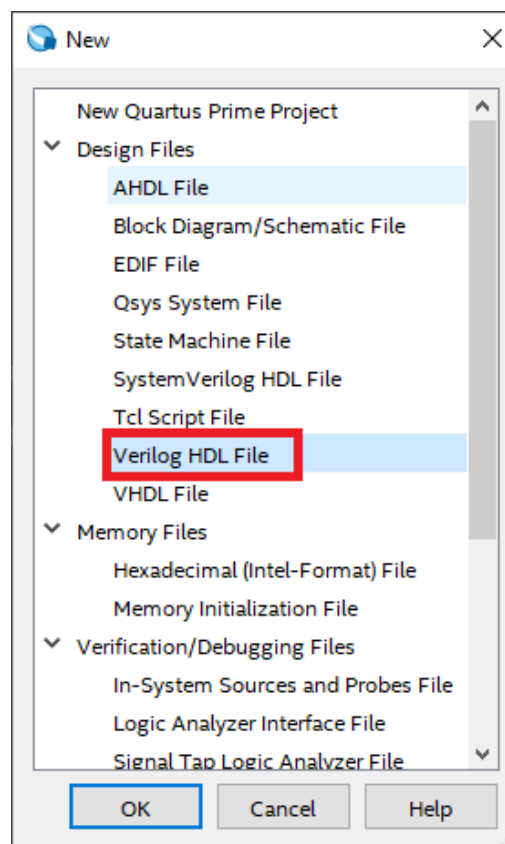
Hình 35. Kết quả sau khi thêm file.

- Sau khi chọn OK, sẽ trở về giao diện của phần mềm Quartus Prime, file cần thêm đã xuất hiện như hình 36.



Hình 36. Kết quả sau khi thêm file ở giao diện Quartus .

- Tiếp theo, tạo file top-level cho project. Chọn **File** → **New (hoặc Ctrl + N)**, sẽ có cửa sổ hiện ra như hình 37. Chọn file mới là Verilog HDL File như hình 37.



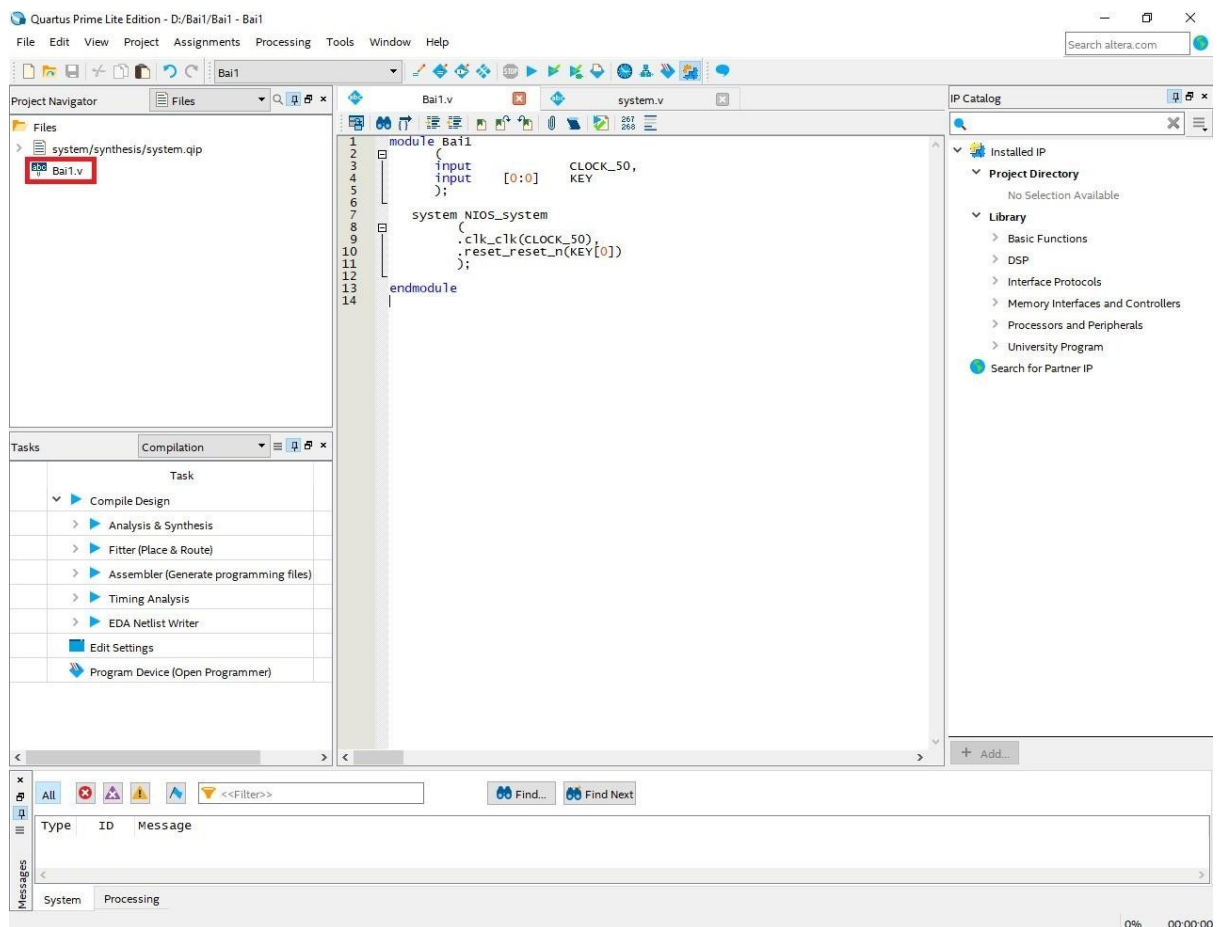
Hình 37. Cửa sổ New.

- o Gõ đoạn code bên dưới và lưu vào project như hình 38.

```
module Bail
(
    input          CLOCK_50,
    input [0:0]    KEY
);

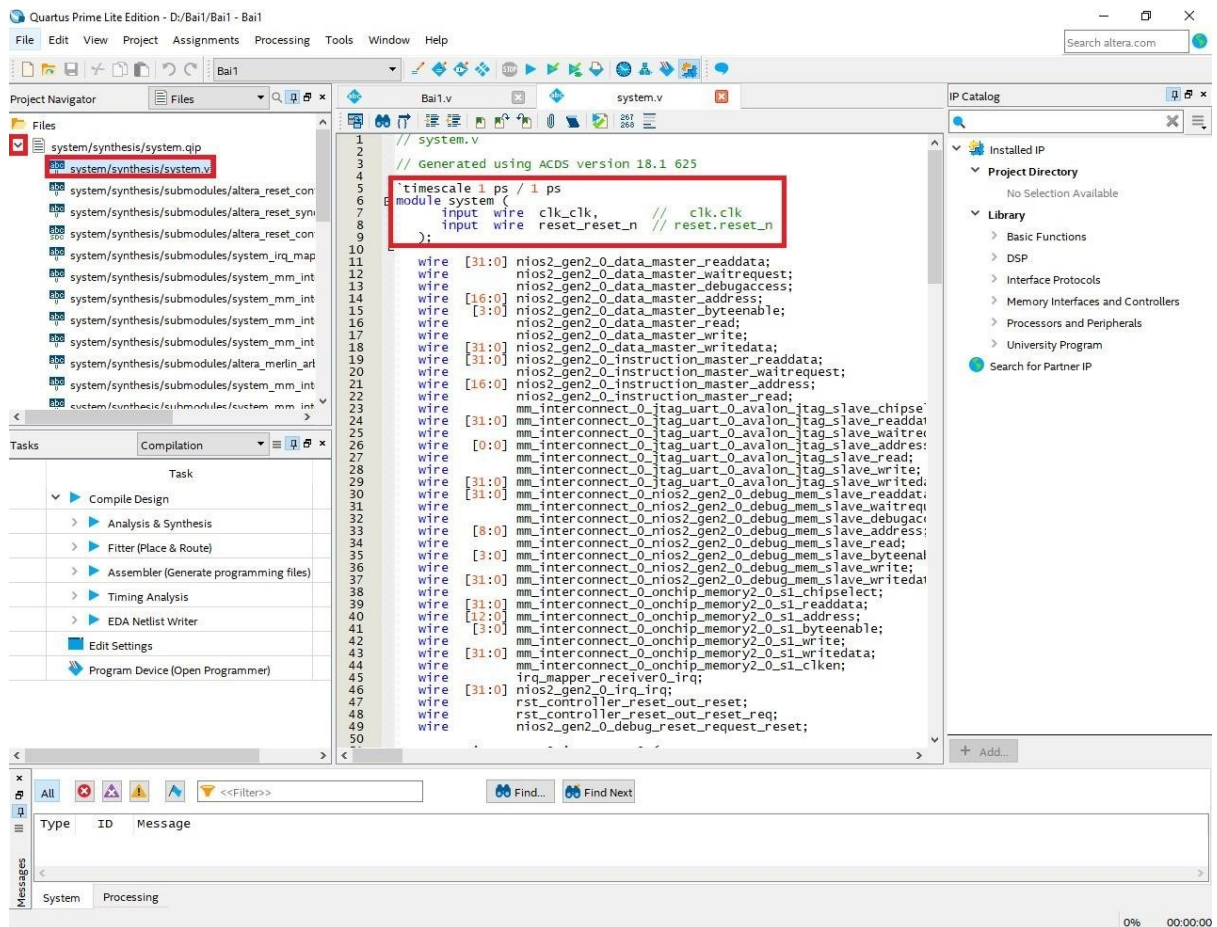
    system NIOS_system
    (
        .clk_clk(CLOCK_50),
        .reset_reset_n(KEY[0])
    );

endmodule
```



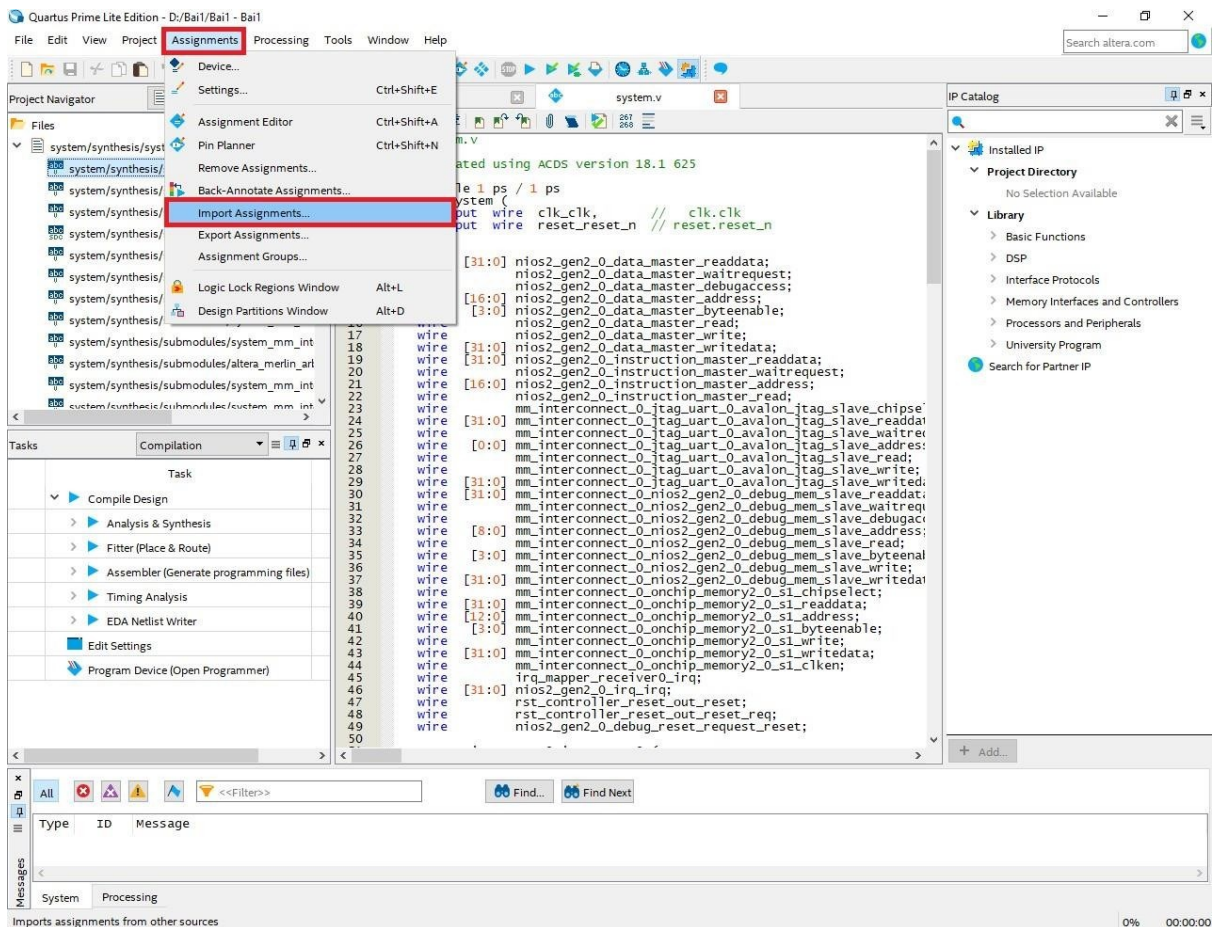
Hình 38. Giao diện Quartus sau khi thêm file Bai1.v

- Để biết được các kết nối vào ra của mô đun system, mở file **system.v**. Hình 39 thể hiện code và các kết nối vào ra của mô đun system.



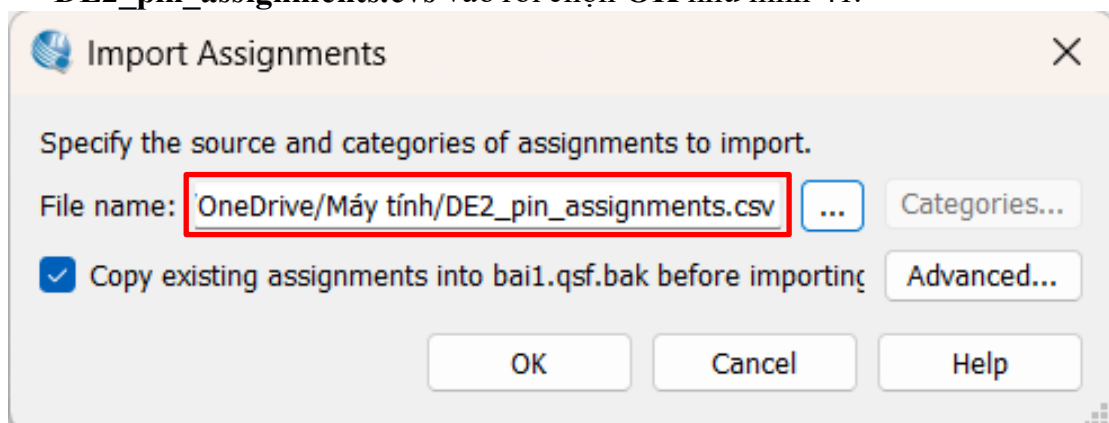
Hình 39. Các chân tín hiệu của mô đun system.

- Sau đó, tiến hành gán pin cho các tín hiệu, chọn **Assignment → Import assignments ...** như hình 40.



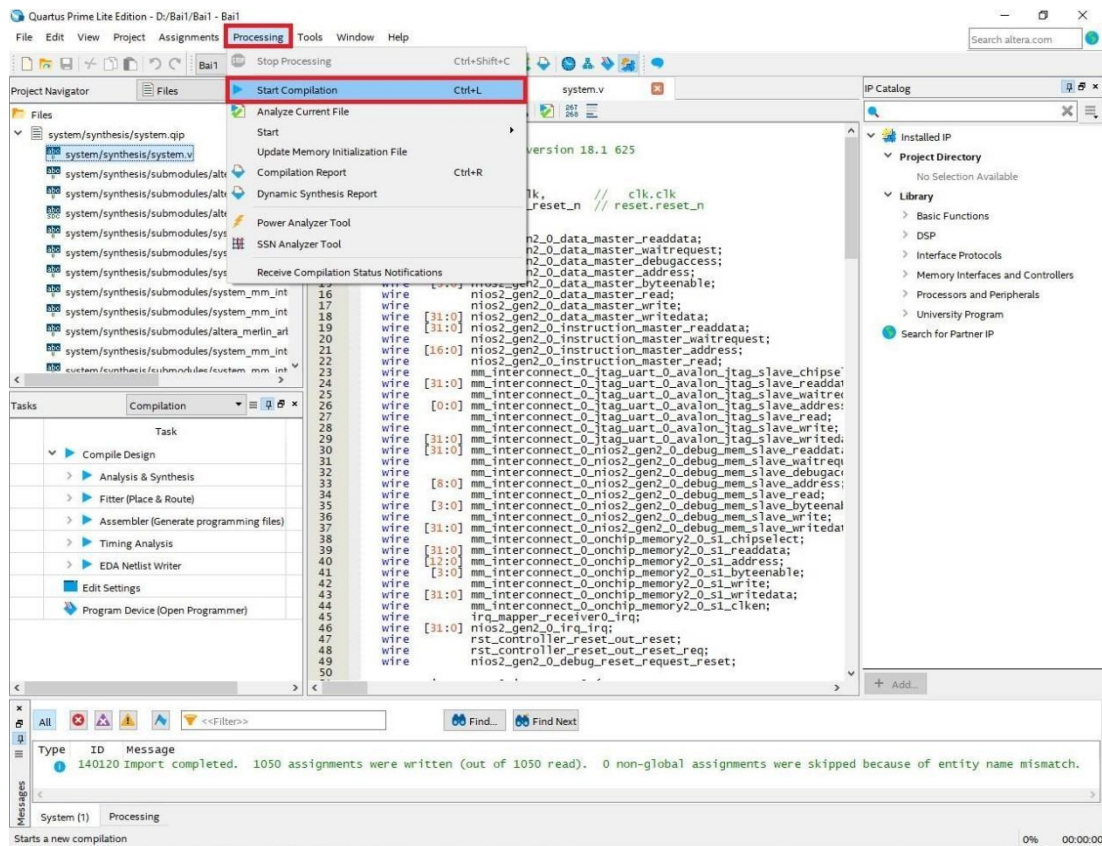
Hình 40. Gán chân cho thiết bị.

- Nếu sử dụng board DE2: Chọn nút ... và dẫn đường dẫn file **DE2_pin_assignments.csv** vào rồi chọn **OK** như hình 41.



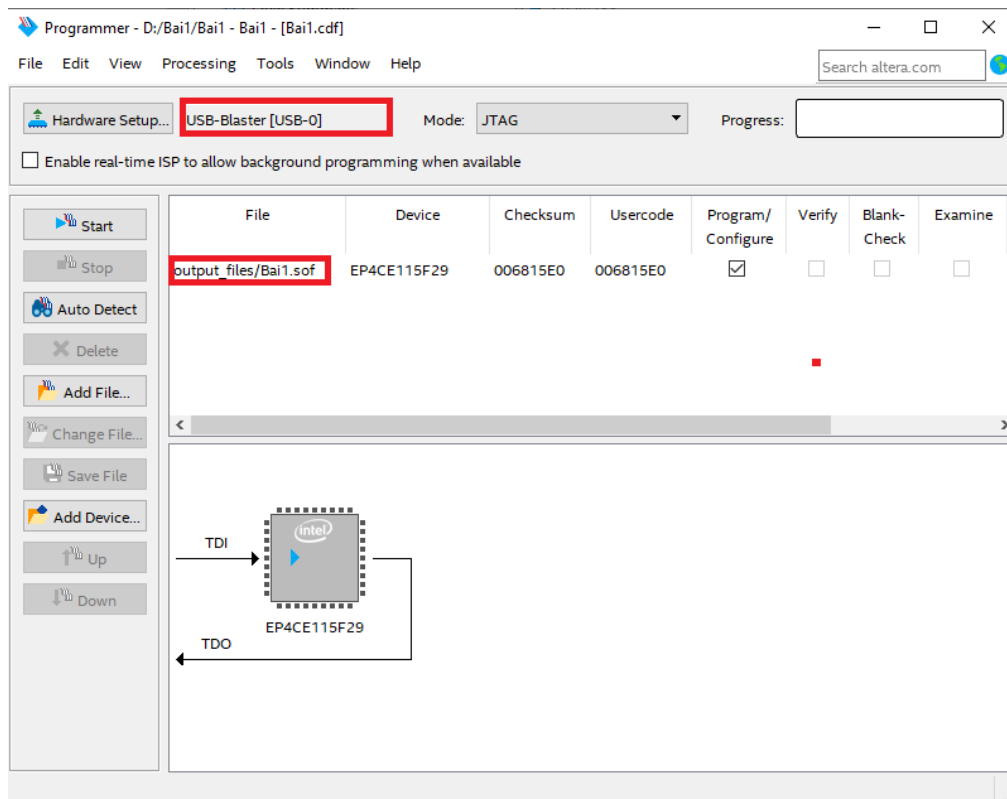
Hình 41. Cửa sổ Import Assignments.

- Cuối cùng, tổng hợp phần cứng bằng cách chọn **Processing** → **Start Compilation** (hoặc **Ctrl + L**), hình 43.



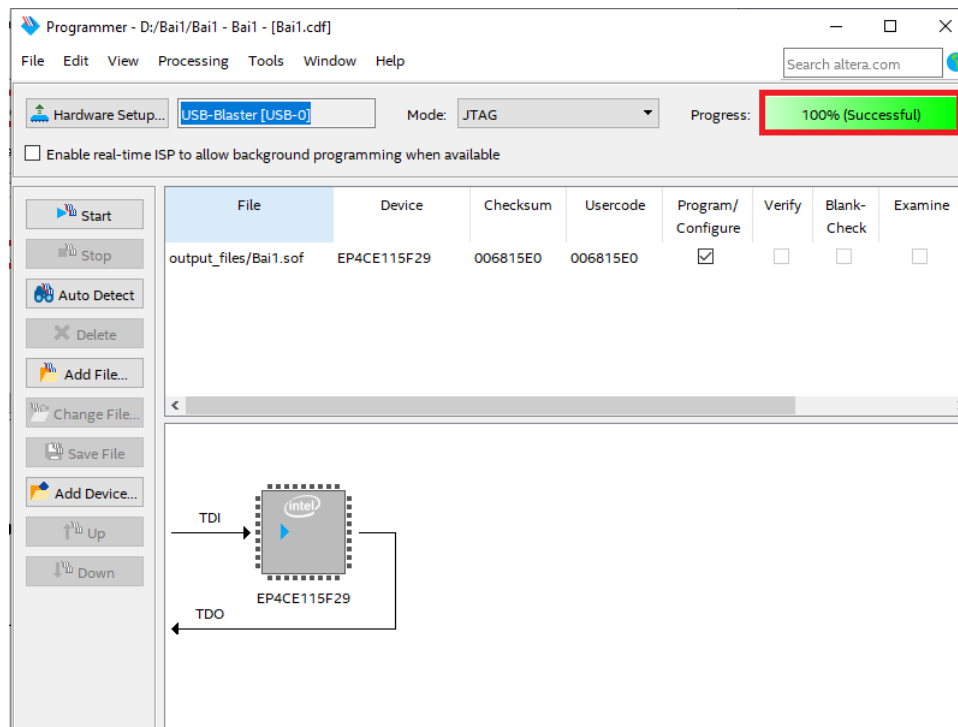
Hình 43. Tiến hành biên dịch.

- Sau khi biên dịch hoàn tất, nạp phần cứng xuống board DE2, chọn Tools → Programmer. Một cửa sổ hiện ra, chọn như hình 44. Yêu cầu là phải có **USB- Blaster [USB-0]** và **Bai1.sof** như trong hình 44. Có thể ở mục device sẽ khác nhau tùy thuộc vào loại board đang sử dụng



Hình 44. Cửa sổ Programmer.

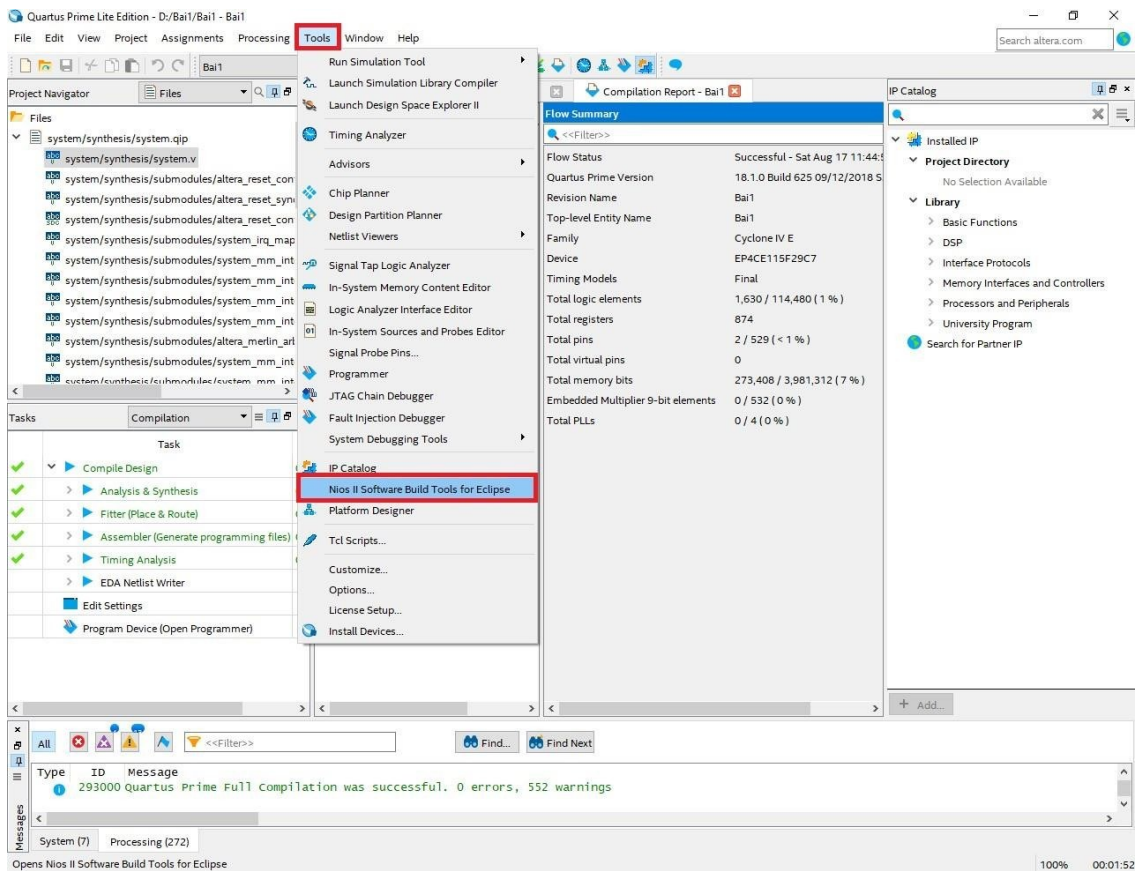
- Sau đó, chọn Start để nạp phần cứng xuống board. Nếu nạp thành công thì các LED sẽ tắt hết. Quá trình nạp thành công được thông báo như hình 45.



Hình 45. Kết quả nạp phần cứng xuống board.

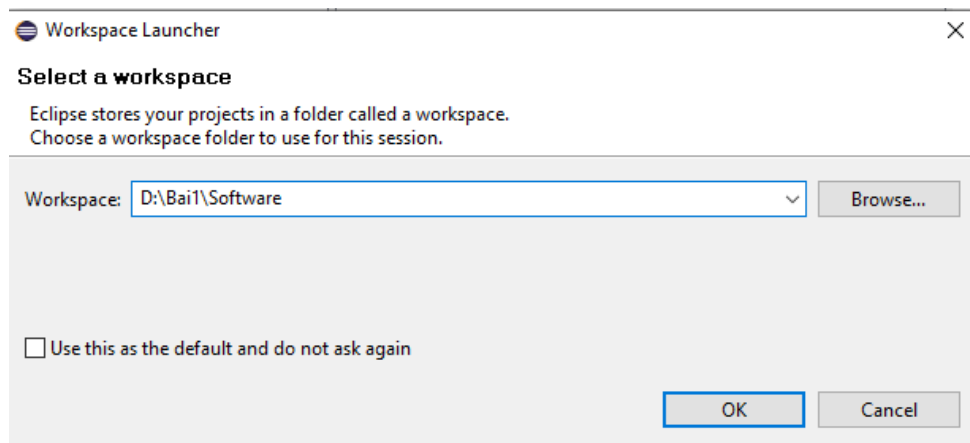
2.2. Xây dựng phần mềm

- Khởi động phần mềm NIOSII bằng cách chọn **Tools** → **NIOS II Software Build Tools for Eclipse**, hình 46.



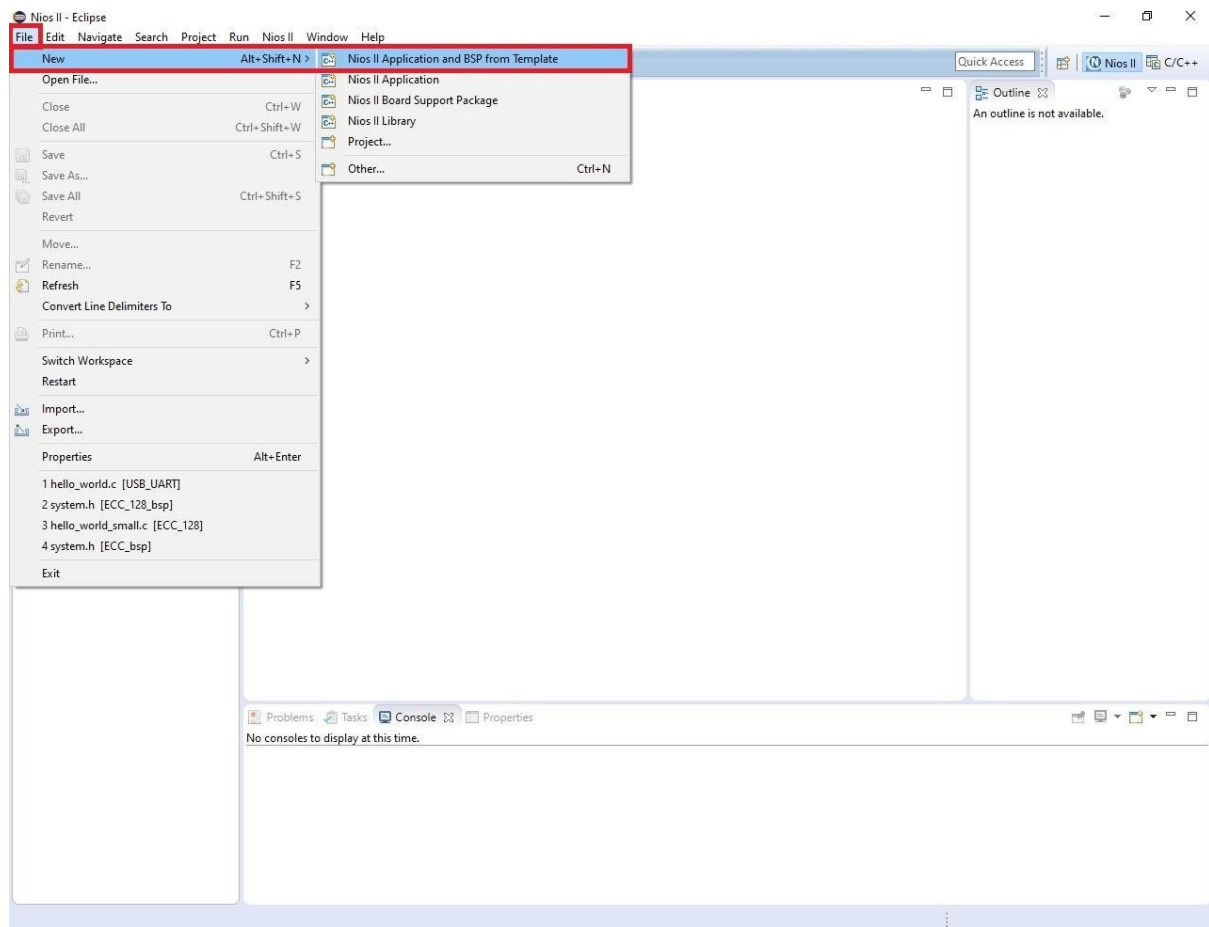
Hình 46. Mở phần mềm Nios II

- Sau khi mở xong Elipse, tạo workspake như hình 47, lưu ý không tạo đường dẫn có khoảng trắng.



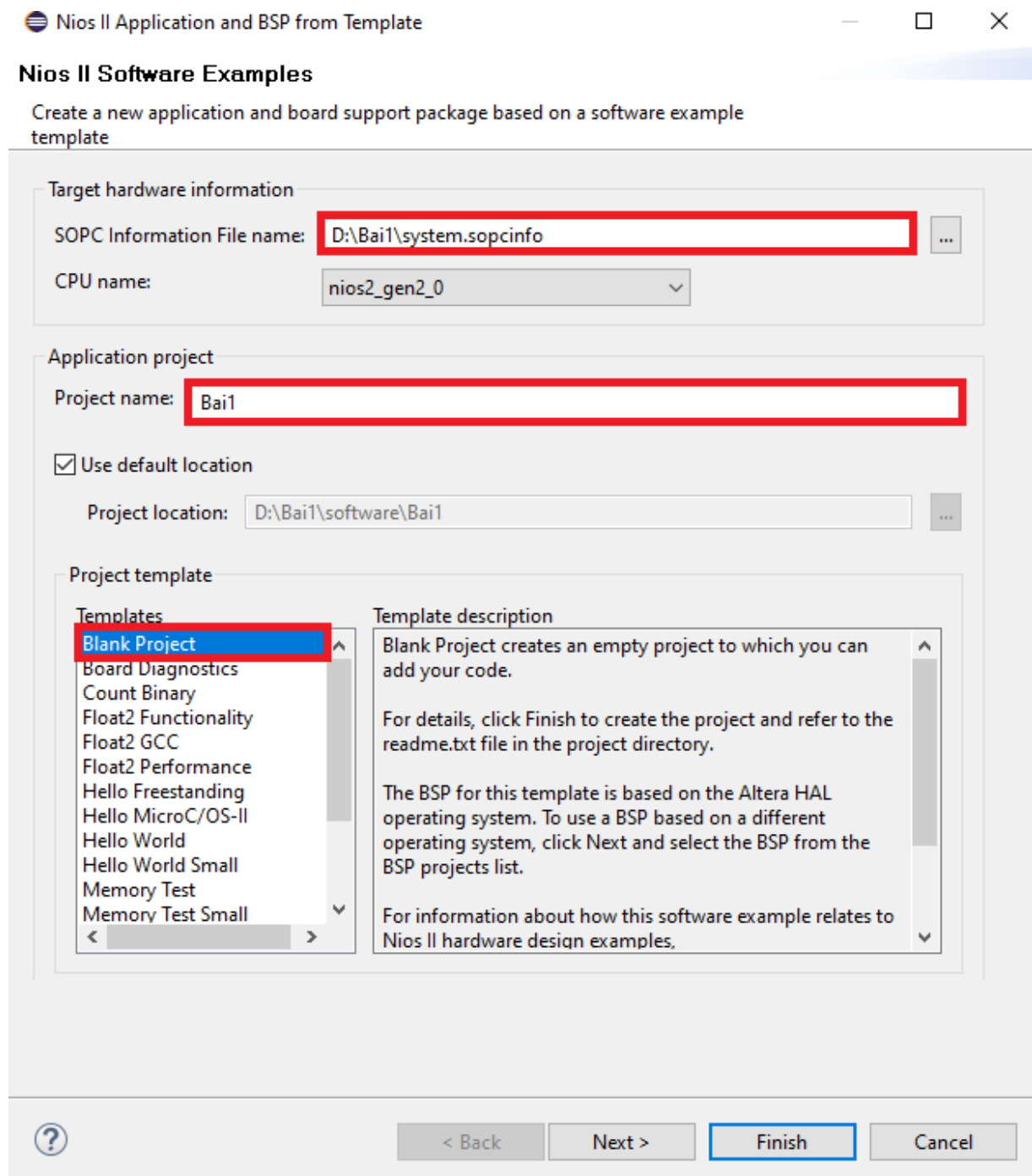
Hình 47. Tạo workspake.

Tiến hành tạo project như hình 48.



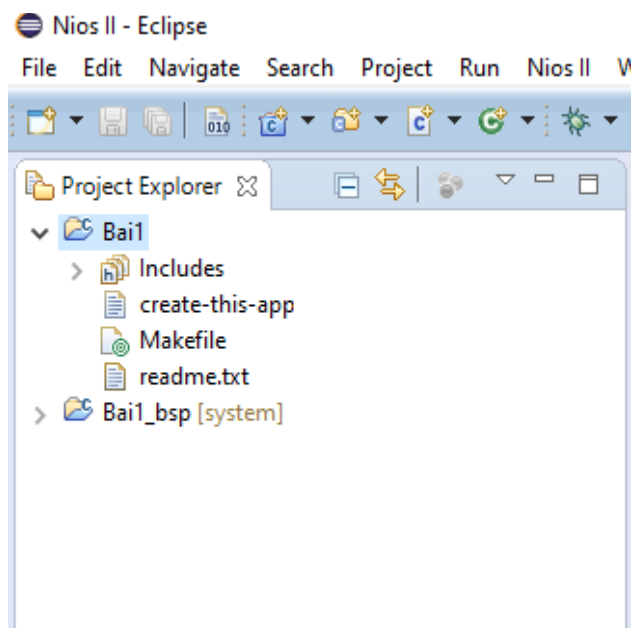
Hình 48. Tạo project.

- Ở cửa sổ Nios II Software Examples, thiết lập như hình 49 rồi chọn **Finish**.



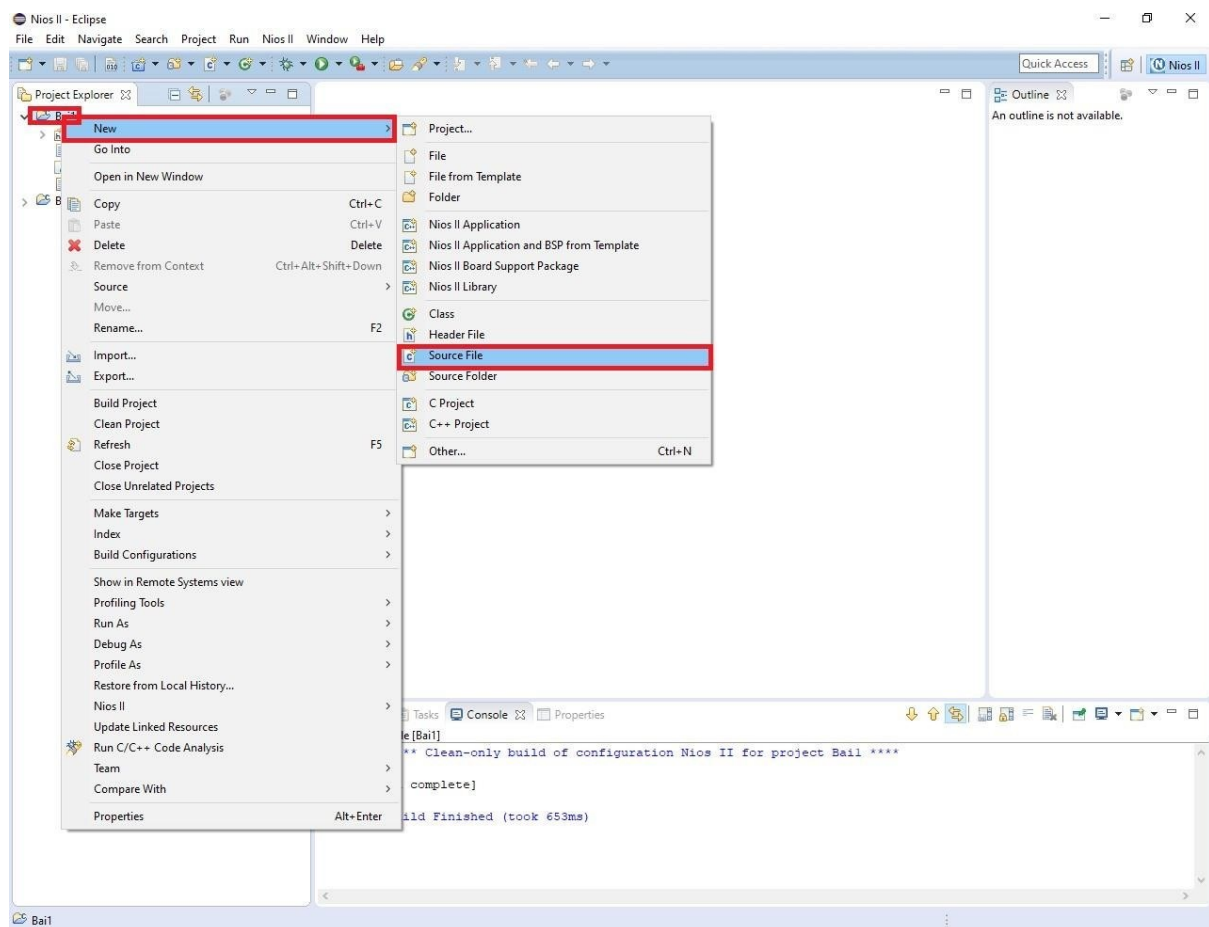
Hình 49. Cửa sổ Nios II Software Examples.

- Sau khi tạo thành công, sẽ được như hình 50.



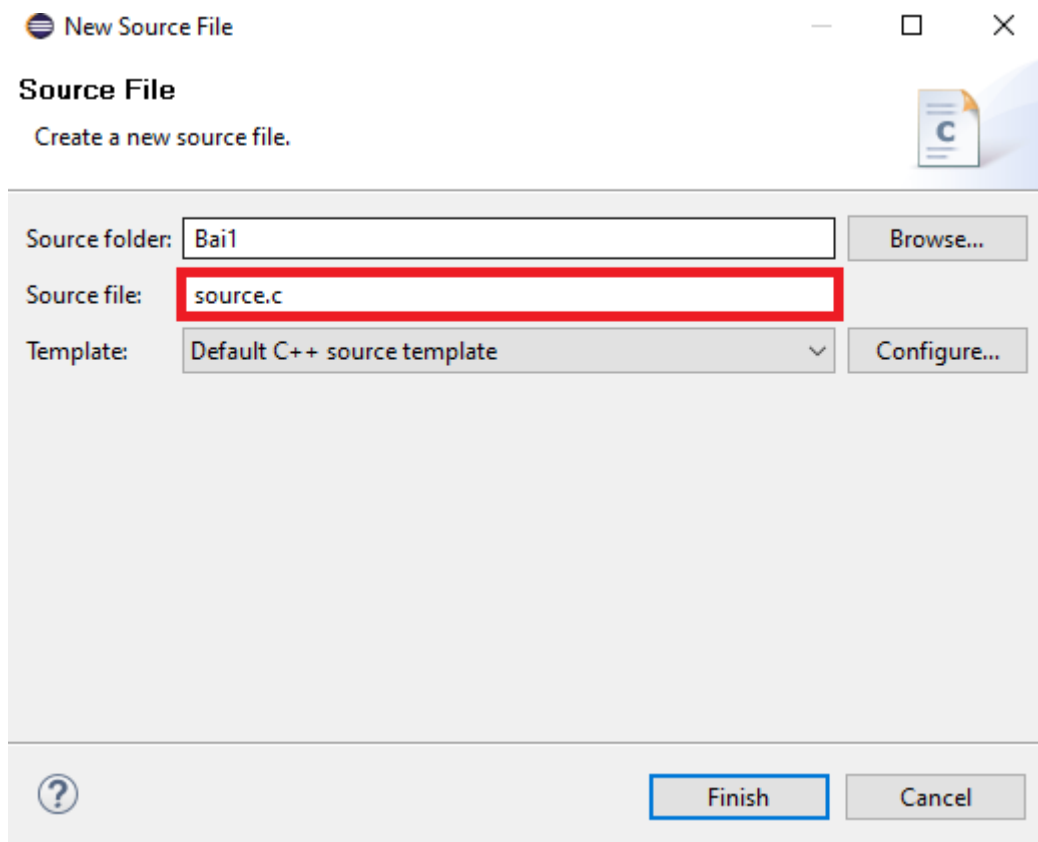
Hình 50. Kết quả sau khi tạo project.

- Tiến hành thêm file source.c như hình 51.



Hình 51. Thêm file source.c

- Đặt tên là file là source.c như hình 52.



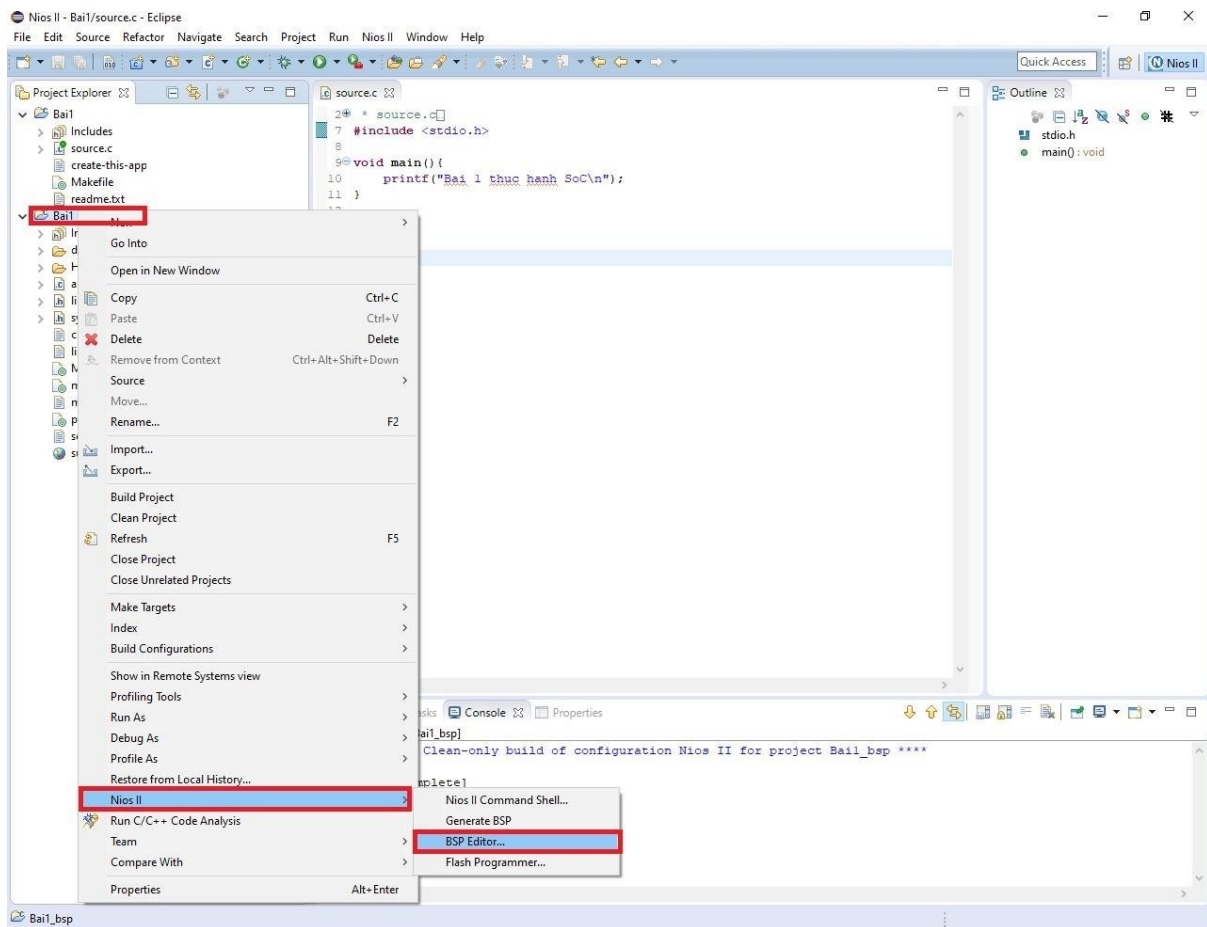
Hình 52. Đặt tên file là *source.c*.

- Tiến hành gõ đoạn code sau vào trong file **source.c** lưu lại.

```
#include <stdio.h>

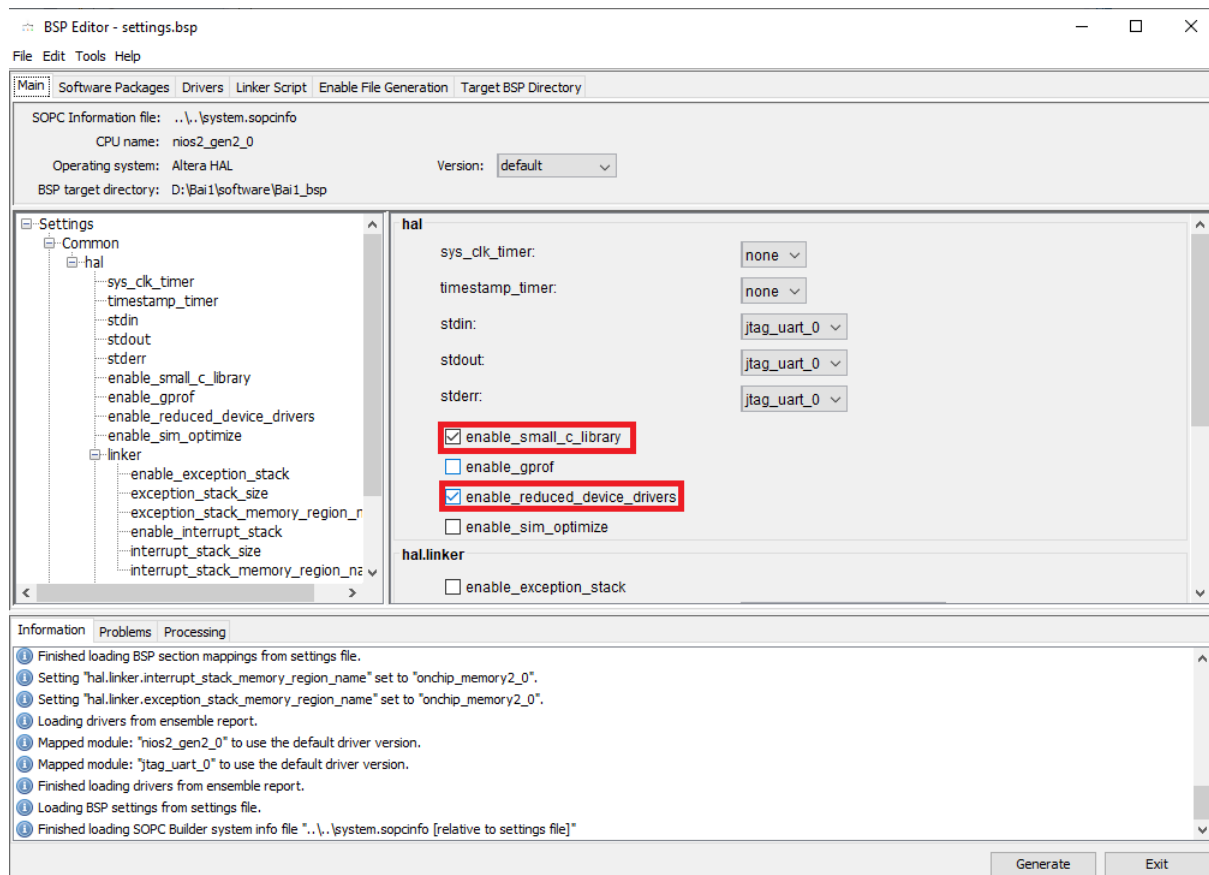
void main() {
    printf("Bai 1 thuc hanh SoC\n");
}
```

- Cấu hình lại bsp, chọn chuột phải vào Bai1_bsp, chọn Nios II → BSP Editor... như hình 53.



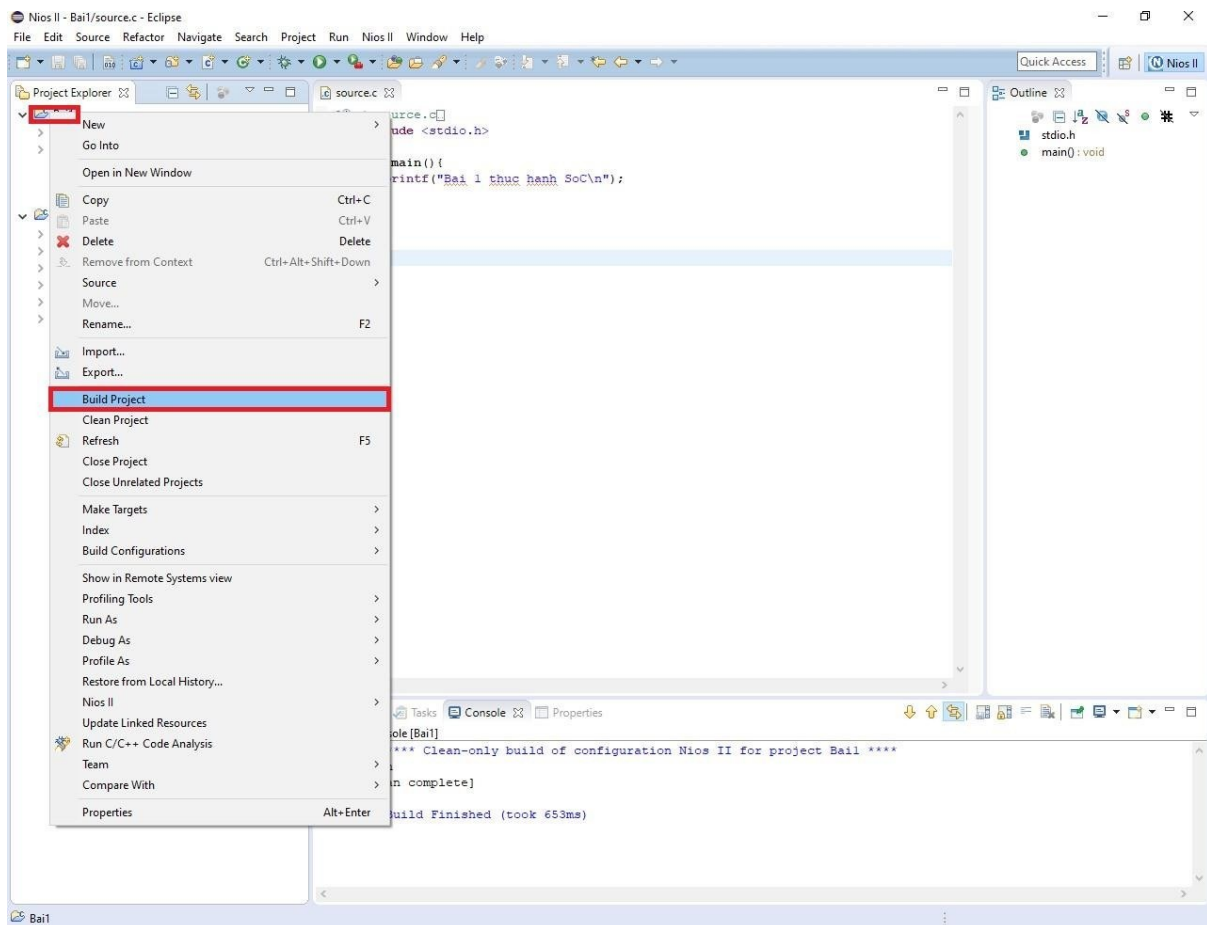
Hình 53. Cấu hình BSP.

- Chọn như hình 54 sau đó chọn **Generate** rồi chọn **Exit**.



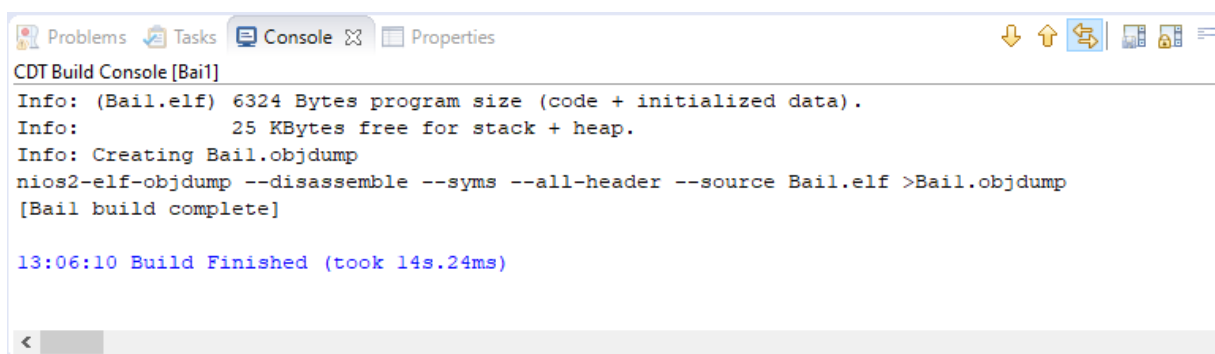
Hình 54. Cấu hình BSP.

- Tiến hành Build project, chọn chuột phải vào **Bai1** → **Build Project** như hình 55.



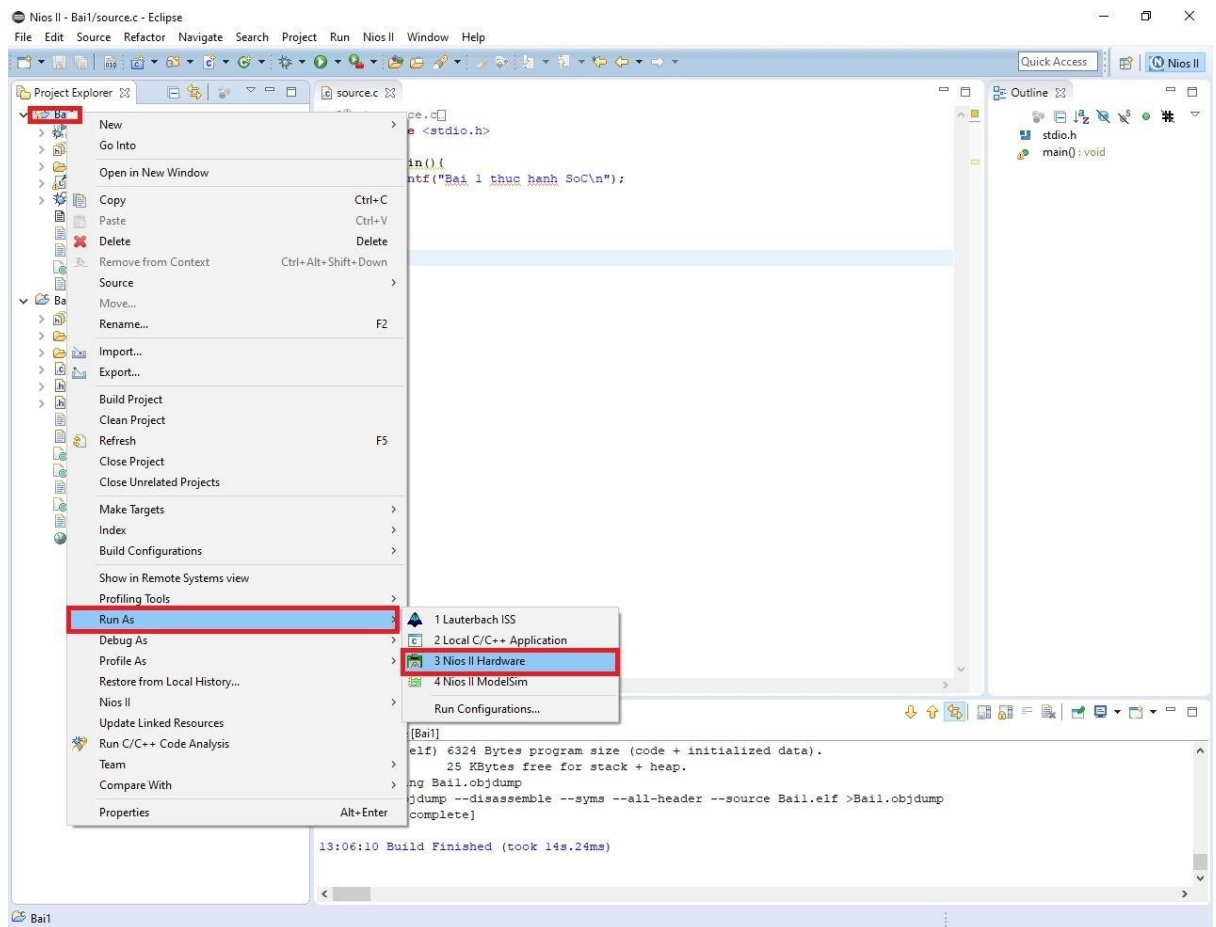
Hình 55. Build project.

- Khi kết quả không có lỗi như hình 56, tiến hành nạp phần mềm xuống board như hình 57.



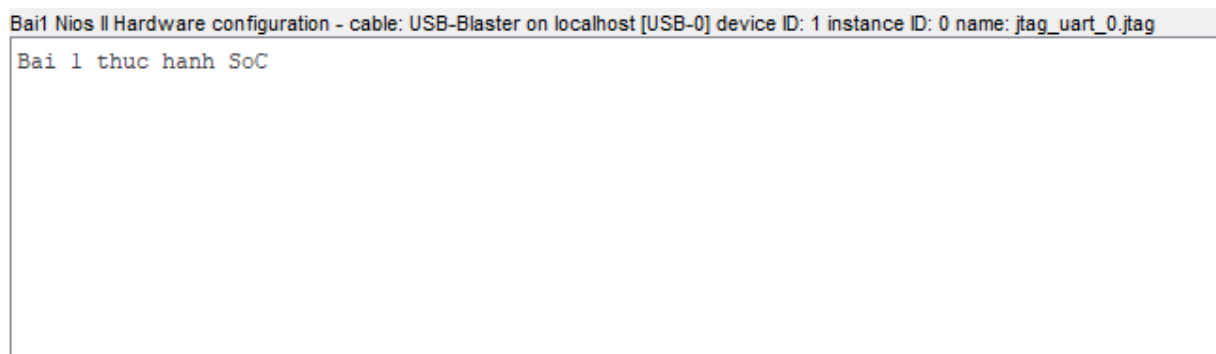
Hình 56. Kết quả build project.

- Nạp phần mềm xuống board bằng cách chọn chuột phải ở **Bai1** → **Run as** → **Nios II Hardware**, hình 57.



Hình 57. Nạp phần mềm xuống board.

- Kết quả của sau khi nạp phần mềm được thể hiện như hình 58.



Hình 58. Kết quả chạy trên board hiển thị trên console.

BÀI TẬP CHUẨN BỊ Ở NHÀ

Bài 1. Cho biết thật hống SoC là gì?

Bài 2. Dựa vào tài liệu tham khảo [2], cho biết chức năng của mô đun Jtag-uart.

BÁO CÁO THỰC HÀNH

Bài 1. Tìm hiểu cách Debug và cho biết ý nghĩa của và phím tắt của các biểu tượng sau trong cửa sổ debug của NIOS II:



Bài 2: Tiến hành debug đoạn code sau:

```
#include <stdio.h>

void main() {
    int a;
    int *b;
    int c;
    a = 3;
    b = &a;
    *b = 5;
    c = a + 4;
}
```


BÀI 2. GIAO TIẾP I/O ĐƠN GIẢN

1. Mục tiêu

Thông qua bài thực hành này, sinh viên sẽ hiểu rõ:

- Cấu trúc bộ PIO (Parallel Input/Output) của Intel (các thanh ghi, chức năng ...).
- Cách xây dựng hệ thống phần cứng với PIO dùng Platform Designer để thực hiện chức năng I/O đơn giản.
- Cách truy xuất tới khối phần cứng PIO đã thiết kế trên chương trình C thông qua Nios II - Eclipse.

2. Phần lý thuyết

2.1. Giới thiệu

- Khối PIO (được cung cấp sẵn bởi Intel) có chức năng thực hiện các giao tiếp IO, thu nhận dữ liệu từ (switches, nút nhấn, ...) hoặc xuất dữ liệu ra ngoài (leds đỏ, leds xanh, ...).
- Module PIO có thể được cấu hình để hoạt động ở một trong bốn chế độ sau:
 - Input: Thu nhận dữ liệu từ các tín hiệu bên ngoài.
 - Output: Xuất dữ liệu ra các tín hiệu bên ngoài
 - Bidir: Vừa thu nhận dữ liệu, vừa xuất dữ liệu (không đồng thời)
 - InOut: Vừa thu nhận dữ liệu, vừa xuất dữ liệu (đồng thời).
- CPU Nios sẽ điều khiển khối PIO thông qua cách đọc hoặc ghi lên các thanh ghi của PIO thông qua giao tiếp chuẩn Avalon Bus.
- Độ dài các thanh ghi n từ 1 đến 32 bits và hướng của dữ liệu được cấu hình bởi người dùng thông qua công cụ Qsys.

2.2. Thanh ghi của PIO

- PIO bao gồm 4 thanh ghi chính được mô tả ở bảng 1.

Offset	Register		R/W	Description
0	data	read access	R	Data value currently on PIO inputs
		write access	W	New value to drive on PIO outputs
1	direction		R/W	Individual direction control for each I/O port. A value of 0 sets the direction to input; 1 sets the direction to output.
2	interruptmask		R/W	IRQ enable/disable for each input port. Setting a bit to 1 enables interrupts for the corresponding port.
3	edgecapture		R/W	Edge detection for each input port.

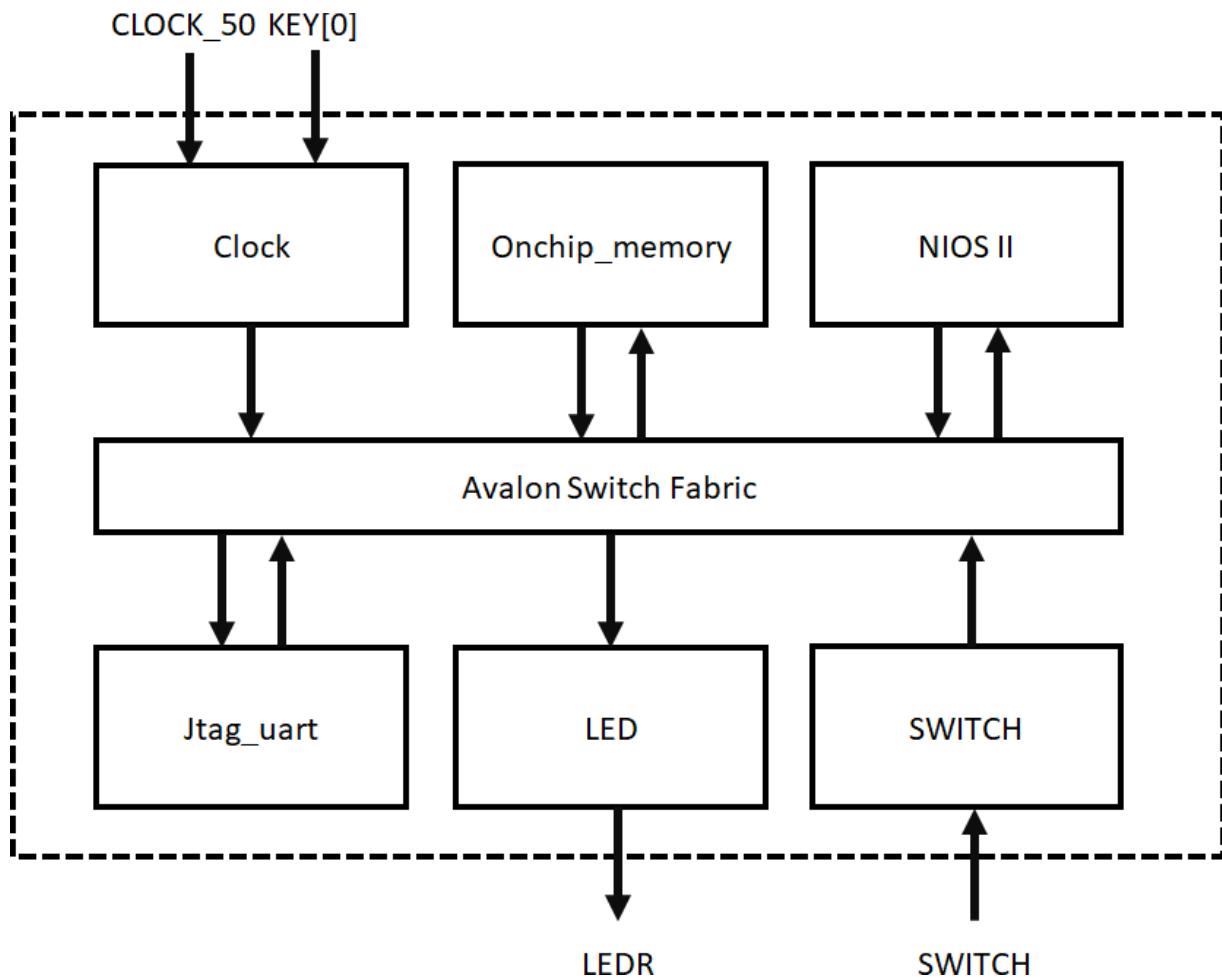
4	outset	W	Specifies which bit of the output port to set. Outset value is not stored into a physical register in the IP core. Hence it's value is not reserve for future use.
5	outclear	W	Specifies which output bit to clear. Outclear value is not stored into a physical register in the IP core. Hence it's value is not reserve for future use.

- Chức năng của mỗi thanh ghi:
 - **Data Register**: thanh ghi chứa dữ liệu thu vào (chế độ Input) hoặc xuất ra (chế độ Output).
 - **Direction Register**: thanh ghi hướng của dữ liệu (chế độ Bidir).
 - **Interrupt-mask Register**: thanh ghi cho phép ngắt (chế độ Input).
 - **Edge-capture Register**: thanh ghi báo hiệu có cạnh lên/cạnh xuống của tín hiệu ngõ vào (chế độ Input).
- Mỗi thanh ghi của PIO có thể được truy xuất như một vị trí nhớ trong bộ nhớ. Địa chỉ để truy xuất các thanh ghi sẽ là tổng của Base Address và Offset.
 - ✓ Base Address: địa chỉ của khối PIO. Base Address được gán tự động thông qua Platform Designer.
 - ✓ Offset: Vị trí tương đối của các thanh ghi so với thanh ghi đầu tiên (Data Register). Địa chỉ của thanh ghi Direction, Interrupt-mask và Edge-capture sẽ có offset lần lượt là 4, 8, và 12 (trong trường hợp thanh ghi PIO là 32 bits hay 4 bytes).
 - ✓ Địa chỉ cuối cùng của từng thanh ghi sẽ theo công thức: Address = Base Address + Offset.
 - ✓ Ví dụ: PIO 32 bit được SoPC Builder gán Base Address là 0x11000 → địa chỉ thanh ghi Data là 0x11000 (offset = 0), của thanh ghi Direction là 0x11004 (offset = 4), của Interrupt-mask là 0x11008 (offset = 8) và thanh ghi Edge-capture sẽ có địa chỉ 0x1100c (offset = 12).

3. Phần thực hành

3.1. Tổng quan hệ thống

- Trong giao diện phần mềm Qsys, ta xây dựng phần cứng như hình 1.



Hình 1. Tổng quan hệ thống.

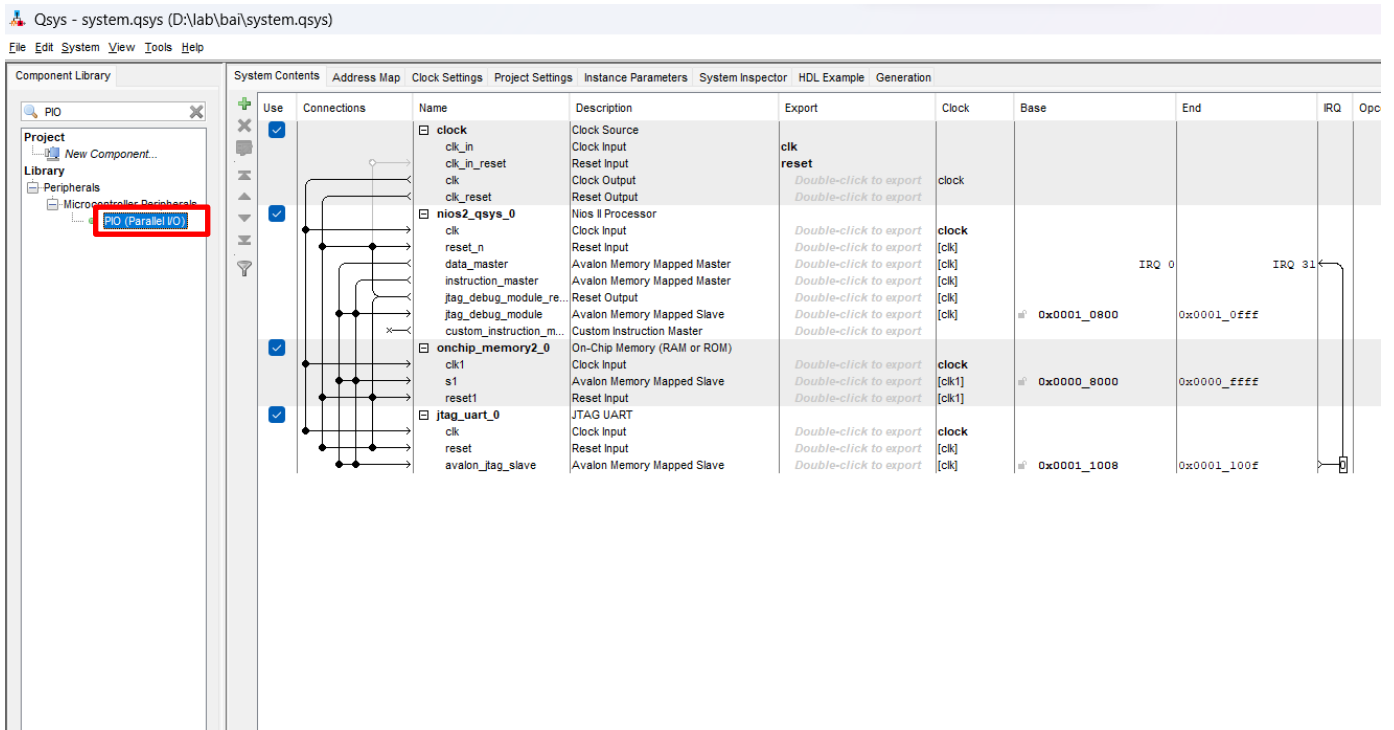
3.2. Xây dựng phần cứng

3.2.1. Tạo project

- Tạo project theo hướng dẫn ở bài 1. Trong đó, tạo project Quartus tên là “lab2”. Lưu ý đường dẫn thư mục project không được có khoảng trắng.
- Chọn Family là *Cyclone II* và device name là *EP2C35F672C6* như hình 6 nếu dùng board *DE2*.

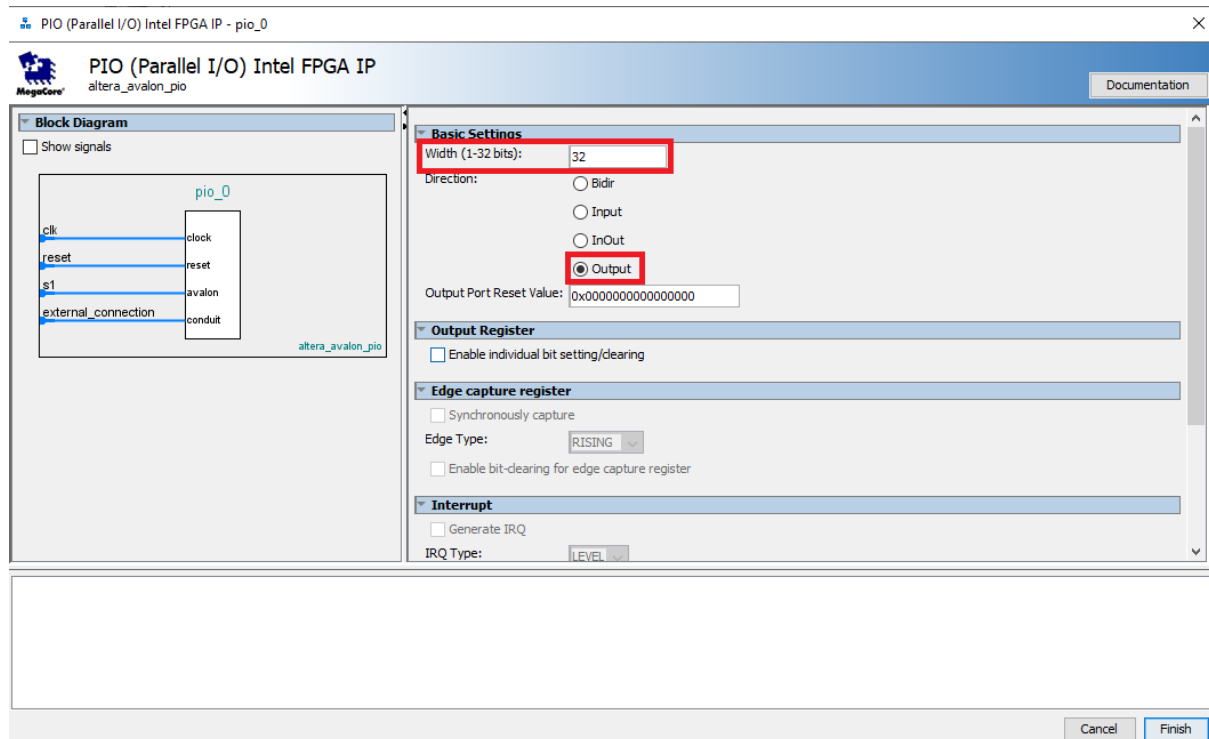
3.2.2. Xây dựng hệ thống phần cứng

- Xây dựng phần cứng theo quy trình ở bài 1. Trong bài này chỉ trình bày ngắn gọn quy trình tạo phần cứng trên Qsys, tạo file top-level và tổng hợp phần cứng.
- Trên hệ thống phần cứng như bài 1, thêm mô đun PIO bằng cách gõ PIO vào ô tìm kiếm và nhấp đúp chuột trái để thêm mô đun vào hệ thống, hình 2.



Hình 2. Thêm mô đun PIO.

- Khi thêm mô đun PIO, sẽ xuất hiện hộp thoại cấu hình PIO, cấu hình là độ rộng dữ liệu là 32 bits, hướng là output, hình 3.



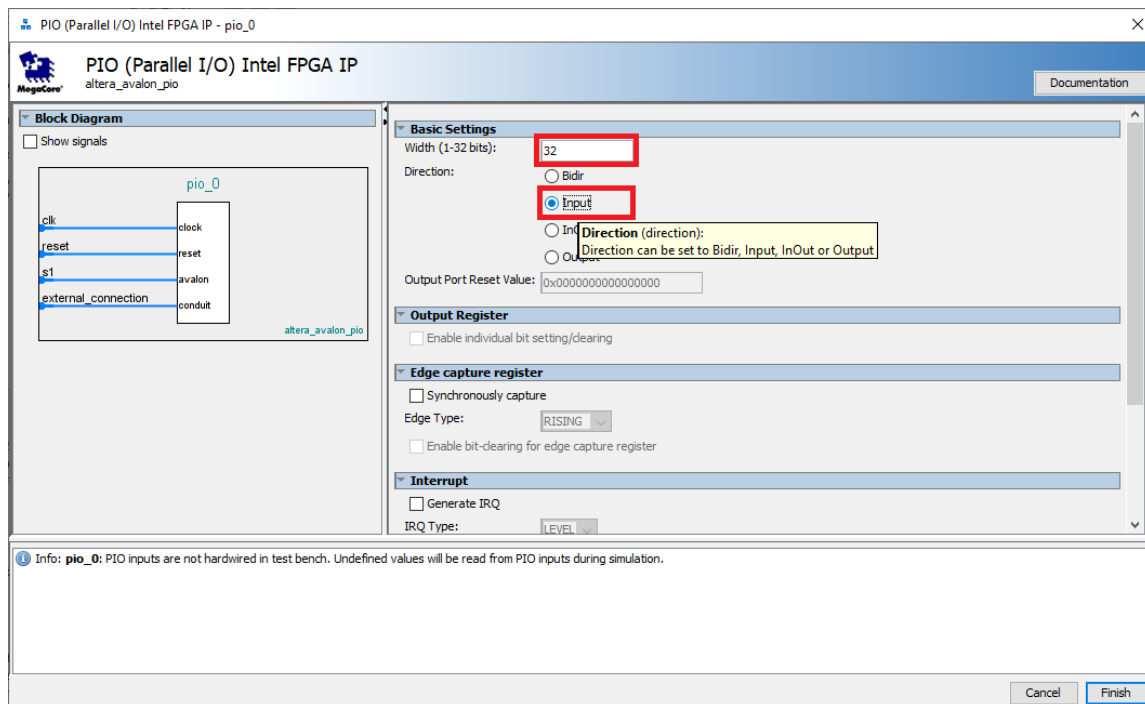
Hình 3. Cấu hình mô đun PIO.

- Sau khi thêm, đổi tên pio_0 thành **led** và kết nối vào hệ thống như hình 4. Lưu ý ở cột Export, ta nhấp đúp chuột trái để xuất tín hiệu ra ngoài.

System Contents	Address Map	Clock Settings	Project Settings	Instance Parameters	System Inspector	HDL Example	Generation			
Use	Connections	Name	Description	Export	Clock	Base	End	IRQ	Opcode Name	
		clock	Clock Source	clk						
		clk_in_reset	Clock Input	reset						
		clk_in_reset	Reset Input	clk						
		clk	Clock Output	Double-click to export	clock					
		clk_reset	Reset Output	Double-click to export						
		nios2_qsys_0	Nios II Processor							
		clk	Clock Input	Double-click to export	clock					
		reset_n	Reset Input	Double-click to export	[clk]					
		data_master	Avalon Memory Mapped Master	Double-click to export	[clk]			IRQ 0		
		instruction_master	Avalon Memory Mapped Master	Double-click to export	[clk]					
		jtag_debug_module_re...	Reset Output	Double-click to export	[clk]					
		jtag_debug_module	Avalon Memory Mapped Slave	Double-click to export	[clk]					
		custom_instruction_m...	Custom Instruction Master	Double-click to export	[clk]					
		onchip_memory2_0	On-Chip Memory (RAM or ROM)							
		clk1	Clock Input	Double-click to export	clock					
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk1]					
		reset1	Reset Input	Double-click to export	[clk1]					
		jtag_uart_0	JTAG UART							
		clk	Clock Input	Double-click to export	clock					
		reset	Reset Input	Double-click to export	[clk]					
		avalon_jtag_slave	Avalon Memory Mapped Slave	Double-click to export	[clk]					
		led	PIO (Parallel I/O)	Double-click to export	clock					
		clk	Clock Input	Double-click to export	[clk]					
		reset	Reset Input	Double-click to export	[clk]					
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk]					
		external_connection	Conduit	led_external_connection						
		switch	PIO (Parallel I/O)	Double-click to export	clock					
		clk	Clock Input	Double-click to export	[clk]					
		reset	Reset Input	Double-click to export	[clk]					
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk]					
		external_connection	Conduit	switch_external_connect...						

Hình 4. Kết quả thêm mô đun PIO-led.

- Tương tự, thêm một mô đun PIO nữa với độ rộng là 32 bits, hướng là input và đặt tên là **switch**. Hình 5 thể hiện việc cấu hình và hình 6 là kết quả của việc thêm mô đun vào hệ thống.

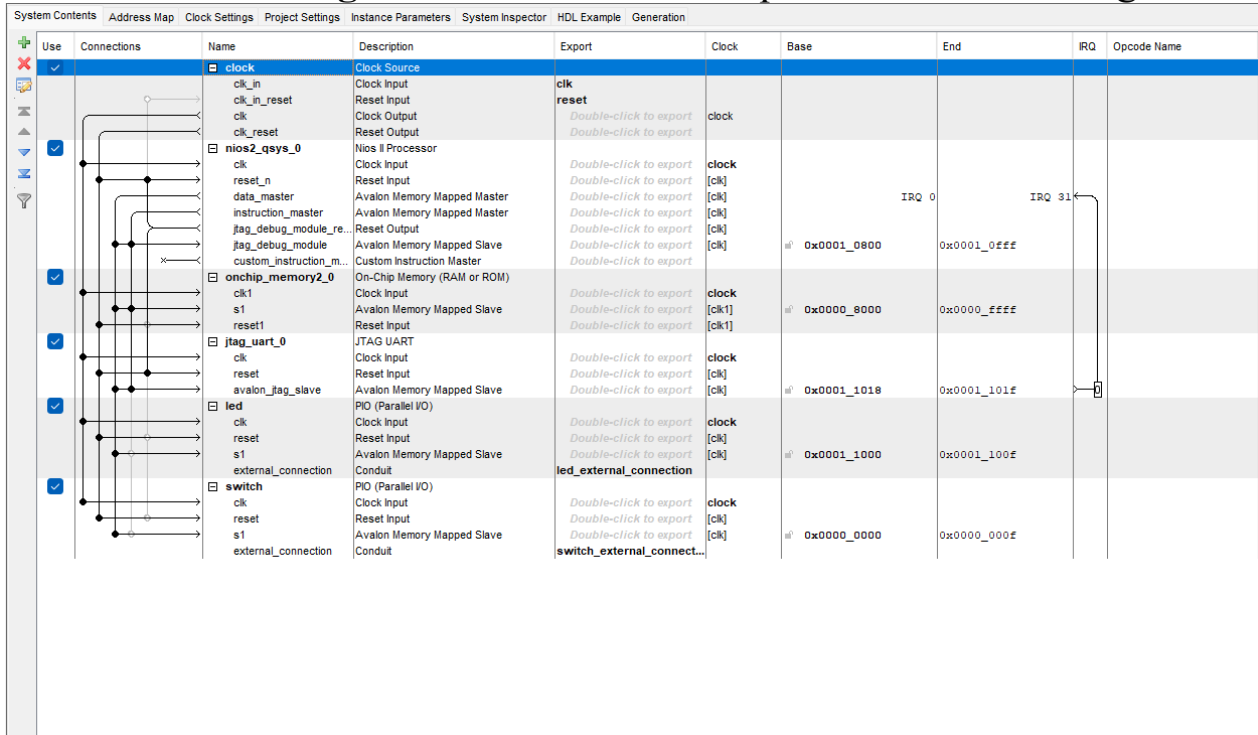


Hình 5. Cấu hình khối PIO-switch.

System Contents	Address Map	Clock Settings	Project Settings	Instance Parameters	System Inspector	HDL Example	Generation
Use							

Hình 6. Kết quả toàn hệ thống.

- Tương tự bài 1, tiến hành gán địa chỉ cho các thành phần trong hệ thống: **System 7 Assign Base Addresses**. Kết quả được thể hiện trong hình 7.



Use	Connections	Name	Description	Export	Clock	Base	End	IRQ	Opcode Name
✓		clock	Clock Source						
		clk_in	Clock Input	clk					
		clk_in_reset	Reset Input	reset					
		clk	Clock Output	Double-click to export	clock				
		clk_reset	Reset Output	Double-click to export					
✓		nios2_qsys_0	Nios II Processor						
		clk	Clock Input	Double-click to export	clock				
		reset_n	Reset Input	Double-click to export	[clk]				
		data_master	Avalon Memory Mapped Master	Double-click to export	[clk]				
		instruction_master	Avalon Memory Mapped Master	Double-click to export	[clk]				
		jtag_debug_module_re...	Reset Output	Double-click to export	[clk]				
		jtag_debug_module...	Avalon Memory Mapped Slave	Double-click to export	[clk]	0x0001_0800	0x0001_0fff		
		custom_instruction_m...	Custom Instruction Master	Double-click to export					
✓		onchip_memory2_0	On-Chip Memory (RAM or ROM)						
		clk1	Clock Input	Double-click to export	clock				
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk1]	0x0000_8000	0x0000_ffff		
		reset1	Reset Input	Double-click to export	[clk1]				
✓		jtag_uart_0	JTAG UART						
		clk	Clock Input	Double-click to export	clock				
		reset	Reset Input	Double-click to export	[clk]				
		avalon_jtag_slave	Avalon Memory Mapped Slave	Double-click to export	[clk]	0x0001_1018	0x0001_101f		
✓		led	PIO (Parallel I/O)						
		clk	Clock Input	Double-click to export	clock				
		reset	Reset Input	Double-click to export	[clk]				
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk]	0x0001_1000	0x0001_100f		
		external_connection	Conduit	led_external_connection					
✓		switch	PIO (Parallel I/O)						
		clk	Clock Input	Double-click to export	clock				
		reset	Reset Input	Double-click to export	[clk]				
		s1	Avalon Memory Mapped Slave	Double-click to export	[clk]	0x0000_0000	0x0000_000f		
		external_connection	Conduit	switch_external_connect...					

Hình 7. Hệ thống với các địa chỉ Base.

- Ta để ý ở cột **Base**, đó là địa chỉ bắt đầu của các mô đun như đã đề cập ở phần lý thuyết.
- Cuối cùng lưu lại và đặt tên hệ thống là **system** và generate hệ thống.

3.2.3. Tích hợp hệ thống.

- Tiến hành thêm file **system.qip** vào project.
- Tiến hành thêm file top-level là **Bai2.v** có nội dung như đoạn code bên dưới.

```
module Bai2
(
    input          CLOCK_50,
    input          [0:0] KEY,
    input          [15:0] SW,
    output         [15:0] LEDR
);

    system nios_system(
        .clk_clk          (CLOCK_50),
        .reset_reset_n   (KEY[0]),
        .switch_external_connection_export ({16'd0,SW}),
        .led_external_connection_export   ({16'd0,LEDR}
    )
);
endmodule
```

- Tiến hành gán pin cho hệ thống bằng **Assignments** → **Import Assignments**.
- Tiến hành biên dịch và nạp phần cứng xuống board.

3.3. Xây dựng phần mềm

- Thực hiện tạo project trên Eclipse tương tự bài 1, đặt tên là Bai2.
- Tạo file **source.c** có nội dung như đoạn code bên dưới, tiến hành build và nạp project phần mềm xuống board, thay đổi switch và quan sát kết quả.

```
#include <io.h>
#include <system.h>

void main() {
    int temp;
    while (1) {
        temp = IORD(SWITCH_BASE, 0);
        IOWR(LED_BASE, 0, temp);
    }
}
```

- Trong đó, **SWITCH_BASE** và **LED_BASE** là địa chỉ base của 2 mô đun **led** và **switch** trong hệ thống phần cứng như được thể hiện trong hình 7. Theo đó, **LED_BASE** là 0x11010 và **SWITCH_BASE** là 0x11000. Hình 8 và hình 9 thể hiện các địa chỉ được define trong file **system.h**.

```
#define ALT_MODULE_CLASS_led altera_avalon_pio
#define LED_BASE 0x11010
#define LED_BIT_CLEARING_EDGE_REGISTER 0
#define LED_BIT_MODIFYING_OUTPUT_REGISTER 0
#define LED_CAPTURE 0
#define LED_DATA_WIDTH 32
#define LED_DO_TEST_BENCH_WIRING 0
#define LED_DRIVEN_SIM_VALUE 0
#define LED_EDGE_TYPE "NONE"
#define LED_FREQ 50000000
#define LED_HAS_IN 0
#define LED_HAS_OUT 1
#define LED_HAS_TRI 0
#define LED_IRQ -1
#define LED_IRQ_INTERRUPT_CONTROLLER_ID -1
#define LED_IRQ_TYPE "NONE"
#define LED_NAME "/dev/led"
#define LED_RESET_VALUE 0
#define LED_SPAN 16
#define LED_TYPE "altera_avalon_pio"
```

Hình 8. Địa chỉ của led.

```
1 #define ALT_MODULE_CLASS_switch altera_avalon_pio
2 #define SWITCH_BASE 0x11000
3 #define SWITCH_BIT_CLEARING_EDGE_REGISTER 0
4 #define SWITCH_BIT_MODIFYING_OUTPUT_REGISTER 0
5 #define SWITCH_CAPTURE 0
6 #define SWITCH_DATA_WIDTH 32
7 #define SWITCH_DO_TEST_BENCH_WIRING 0
8 #define SWITCH_DRIVEN_SIM_VALUE 0
9 #define SWITCH_EDGE_TYPE "NONE"
10 #define SWITCH_FREQ 50000000
11 #define SWITCH_HAS_IN 1
12 #define SWITCH_HAS_OUT 0
13 #define SWITCH_HAS_TRI 0
14 #define SWITCH_IRQ -1
15 #define SWITCH_IRQ_INTERRUPT_CONTROLLER_ID -1
16 #define SWITCH_IRQ_TYPE "NONE"
17 #define SWITCH_NAME "/dev/switch"
18 #define SWITCH_RESET_VALUE 0
19 #define SWITCH_SPAN 16
20 #define SWITCH_TYPE "altera_avalon_pio"
```

Hình 9. Địa chỉ của switch.

BÀI TẬP CHUẨN BỊ Ở NHÀ

Bài 1. Cho biết con trỏ là gì ? Cách truy xuất bộ nhớ với địa chỉ thông qua con trỏ được thực hiện như thế nào ? Ví dụ : ghi giá trị 0xabcd vào địa chỉ : 0x0800 sử dụng con trỏ ?

Bài 2. Giải thích các thanh ghi của bộ PIO của Intel ?

BÁO CÁO THỰC HÀNH

Bài 1. Giải thích cú pháp IORD và IOWR ?

Thay đổi code C trong phần 3.3 để truy xuất bộ led và switch thông qua con trỏ ?

Bài 2. Xây dựng lại phần cứng và phần mềm để tiến hành giải mã LED 7 đoạn được nhập giá trị từ SW[3 :0] và hiển thị trên HEX[0] bằng C code.

TÀI LIỆU THAM KHẢO

- [1] DE2_115_User_Manual.
- [2] Embedded Peripherals IP User Guide.
- [3] Nios II Processor Reference.
- [4] Avalon Interface Specification.

